

**PENGEMBANGAN ARSITEKTUR IOT TERDISTRIBUSI
BERBASIS MICROSERVICES DAN FIRMWARE MODULAR
PADA SISTEM OTOMASI BUDIDAYA AEROPONIK**

TUGAS AKHIR



Oleh

Alif Muhammad Rizky

NIM: 13322006

**PROGRAM STUDI TEKNIK FISIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI BANDUNG
2026**

ABSTRAK

PENGEMBANGAN ARSITEKTUR IOT TERDISTRIBUSI BERBASIS MICROSERVICES DAN FIRMWARE MODULAR PADA SISTEM OTOMASI BUDIDAYA AEROPONIK

Oleh

Alif Muhammad Rizky NIM: 13322006

(Program Studi Teknik Fisika)

Pertanian presisi pada budidaya aeroponik menuntut pemantauan dan kontrol iklimat secara kontinu dan berkecepatan tinggi. Namun, arsitektur monolitik konvensional memiliki keterikatan erat antarkomponen, rentan terhadap kegagalan menyeluruh (*single point of failure*), serta sulit diskalakan secara independen. Penelitian ini merancang, mengimplementasikan, dan mengevaluasi sistem pemantauan dan kontrol aeroponik berbasis arsitektur *microservice* modular pada sisi *server* dan *firmware* modular pada sisi *edge*.

Metode penelitian menggunakan kerangka kerja *Design Science Research* (DSR). Pada sisi *edge*, *firmware* ESP32 berbasis FreeRTOS *dual-core* dirancang dengan pola *factory pattern* dan *protocol registry* untuk mendukung penambahan sensor dinamis melalui berkas konfigurasi tanpa kompilasi ulang. Pada sisi *server*, sistem menerapkan prinsip *database-per-service* yang mengintegrasikan empat layanan inti (Auth, Module, Control, dan Analytics Service) melalui *event bus* NATS JetStream dan API Gateway Kong, dengan protokol MQTT sebagai jalur komunikasi *edge*. Model *Reinforcement Learning* (TD3) dikembangkan dari awal untuk penyesuaian jadwal semprot adaptif berbasis telemetri dan ekstraksi visual kondisi tanaman dari model pra-latih YOLOv8 via REST API.

Pengujian empiris membuktikan modularitas *firmware* berhasil divalidasi melalui integrasi *handler* protokol I2C untuk sensor lingkungan tanpa memodifikasi logika inti program. Pengujian resiliensi (*chaos engineering*) mengonfirmasi pencapaian isolasi kegagalan penuh antar-layanan inti, sementara pengujian beban menunjukkan latensi *end-to-end* tetap di bawah 2 detik pada persentil ke-95. Kebaruan penelitian ini terletak pada penerapan modularitas ganda (*dual-layer modularity*) yang memisahkan dependensi perangkat keras dan komputasi awan. Penelitian ini berkontribusi menyediakan arsitektur referensi IoT pertanian presisi yang tangguh, adaptif, dan mudah dikembangkan.

Kata kunci: pertanian presisi, aeroponik, arsitektur *microservice*, *firmware* modular, event-driven, *database-per-service*, reinforcement learning.

ABSTRACT

DEVELOPMENT OF A DISTRIBUTED IOT ARCHITECTURE BASED ON MICROSERVICES AND MODULAR FIRMWARE FOR AN AEROPONIC CULTIVATION AUTOMATION SYSTEM

By

Alif Muhammad Rizky

NIM: 13322006

(Engineering Physics Program)

Precision agriculture in aeroponic cultivation requires continuous, high-speed microclimate monitoring and control. However, conventional monolithic architectures suffer from tight component coupling, single points of failure, and limited independent scalability. This research aims to design, implement, and evaluate an aeroponic monitoring and control system based on a modular microservice architecture on the server and modular firmware at the edge.

The study employs the Design Science Research (DSR) framework. At the edge, dual-core FreeRTOS ESP32 firmware utilizes the factory pattern and protocol registry for dynamic sensor addition via configuration files without recompilation. On the server, the system applies database-per-service principles across four core services (Auth, Module, Control, Analytics) interconnected via the NATS JetStream event bus and Kong API Gateway, using MQTT for edge communication. A custom Twin Delayed DDPG (TD3) reinforcement learning model is developed for adaptive misting scheduling based on real-time telemetry and visual plant health extracted from a pre-trained YOLOv8 model via REST API.

Testing validates firmware modularity through the addition of an I2C protocol handler without altering core program logic. Chaos engineering tests confirm full fault isolation across core services, while load testing demonstrates end-to-end latency below 2 seconds at the 95th percentile. The novelty lies in the dual-layer modularity paradigm that decouples hardware dependencies from distributed cloud services. This study contributes a robust, adaptive, and scalable IoT reference architecture for precision agriculture.

Keywords: *precision agriculture, aeroponics, microservice architecture, modular firmware, event-driven, database-per-service, reinforcement learning.*

**PENGEMBANGAN ARSITEKTUR IOT TERDISTRIBUSI
BERBASIS MICROSERVICES DAN FIRMWARE MODULAR
PADA SISTEM OTOMASI BUDIDAYA AEROPONIK
HALAMAN PENGESAHAN**

Oleh

Alif Muhammad Rizky NIM: 13322006
(Program Studi Teknik Fisika)

Institut Teknologi Bandung

Menyetujui
Tim Pembimbing

Tanggal

Pembimbing 1

Pembimbing 2

(Nama Pembimbing 2)
NIP. 19xx0x0xxxxxxxxxx

(Nama Pembimbing 2)
NIP. 19xx0x0xxxxxxxxxx

KATA PENGANTAR

DAFTAR ISI

DAFTAR GAMBAR

DAFTAR TABEL

DAFTAR SINGKATAN DAN LAMBANG

Contoh daftar singkatan dan lambang

SINGKATAN	Nama	Pemakaian pertama kali pada halaman
AMR	<i>Adaptive Mesh Refinement</i>	
CT	<i>Computed Tomography</i>	
DNS	Dekomposisi Nilai Singular	
HPLC	<i>High Performance Liquid Chromatography</i>	10
LAMBANG		
A	Konstanta pada hubungan tegangan	
A_l	Contoh simbol	
A_o	Amplitudo sinyal sinar keluar	
A_i	Amplitudo sinyal sinar masuk	
a	Vektor kerapatan foton pada satu elemen	
a_1	Kecepatan	
a_{ij}	Fungsi reaksi variabel dalam koefisien persamaan diferensial	
b	Persamaan dasar perambatan gelombang	
c	Kecepatan sinar	5

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pertanian modern semakin bertumpu pada teknologi untuk meningkatkan efisiensi produksi dan kualitas hasil panen. Salah satu pendekatan yang berkembang pesat adalah **pertanian presisi** (*precision agriculture*), yakni konsep pengelolaan lahan yang memanfaatkan data real-time dari sensor lingkungan untuk mengambil keputusan secara tepat sasaran mulai dari pemberian nutrisi, pengaturan suhu, hingga pengendalian aktuator secara otomatis. Metode aeroponik menjadi salah satu teknik budidaya yang sangat responsif terhadap pendekatan ini, karena akar tanaman tumbuh di udara dan disemprot larutan nutrisi secara berkala; kondisi ini membuat parameter lingkungan seperti suhu, kelembapan, *electrical conductivity* (EC), dan pH larutan menjadi faktor penentu keberhasilan panen yang sensitif dan harus dipantau secara kontinu. Penelitian menunjukkan bahwa pemantauan parameter lingkungan yang tidak akurat dan tidak real-time dapat menyebabkan penurunan hasil panen hingga 40% pada sistem aeroponik [1][2]. Oleh karena itu, dibutuhkan sebuah sistem pemantauan dan kontrol yang tidak hanya mampu membaca data secara otomatis, tetapi juga dapat memberikan respons cepat terhadap perubahan kondisi lingkungan secara presisi dan andal [3].

Namun, membangun sistem IoT untuk pertanian presisi skala nyata bukan perkara mudah. Sistem yang dibuat secara monolitik di mana seluruh fungsi (autentikasi, penyimpanan data, analitik, kontrol aktuator, notifikasi, hingga *machine learning*) dirangkum dalam satu aplikasi besar dikenal memiliki kelemahan serius: sulit dikembangkan, mudah mengalami kegagalan menyeluruh ketika satu komponen bermasalah, dan sangat sulit untuk diskalakan secara selektif [4]. Pendekatan **arsitektur microservice** hadir sebagai solusi: sistem dipecah menjadi layanan-layanan kecil yang mandiri, masing-masing memiliki tanggung jawab tunggal (*single responsibility*), database sendiri (*database-per-service*), dan dapat di-deploy serta dikembangkan secara independen [5]. Penelitian pada *European Journal of Computer Science and Information Technology* (2025) mencatat bahwa 73% organisasi yang mengadopsi microservice menggunakan pola *database-per-*

service untuk mencapai *decoupling* tingkat data, dan implementasi arsitektur berbasis event (*event-driven architecture*) terbukti meningkatkan skalabilitas sistem hingga 71% serta meningkatkan isolasi kegagalan hingga 65% dibandingkan pendekatan *request-response* sinkron [6]. Sistem yang dibangun dalam penelitian ini mengadopsi filosofi ini secara penuh: terdapat 12 layanan backend mandiri yang masing-masing berkomunikasi melalui *event bus* NATS JetStream dan protokol MQTT, dengan total 12 instance database terpisah yang terdiri dari MariaDB, TimescaleDB, Redis, dan MinIO [5][7].

Tantangan utama yang sering muncul dalam pengembangan sistem IoT skala besar adalah **kemudahan pengembangan dan skalabilitas** seiring bertambahnya fitur dan beban kerja. Sistem yang tidak dirancang modular dari awal akan mengalami kesulitan ketika peneliti atau pengembang ingin menambahkan sensor baru, protokol baru, atau kapabilitas *machine learning* tanpa mengganggu fungsi yang sudah berjalan [8]. Dalam penelitian ini, prinsip modularitas diterapkan di dua lapisan sekaligus: (1) pada **sisi firmware** ESP32, di mana setiap jenis sensor dienkapsulasi dalam *handler* mandiri yang dapat ditambah atau dihapus hanya dengan mengubah file konfigurasi `config.json` tanpa kompilasi ulang berkat pola *factory pattern* dan *protocol registry* yang diimplementasikan di atas FreeRTOS dual-core; dan (2) pada **sisi server**, di mana setiap layanan backend berjalan dalam kontainer Docker tersendiri dengan antarmuka komunikasi yang didefinisikan secara eksplisit melalui kontrak NATS subject dan MQTT topic. Pendekatan ini memungkinkan tim pengembang yang berbeda mengerjakan layanan yang berbeda secara paralel, serta memungkinkan skalabilitas selektif hanya layanan yang membutuhkan sumber daya tambahan yang di-*scale*, bukan seluruh sistem [5][9]. Dengan demikian, penelitian ini tidak hanya membangun sistem pemantauan aeroponik, tetapi juga membuktikan secara empiris bahwa arsitektur modular berbasis microservice memberikan keunggulan nyata dalam kemudahan pengembangan, isolasi kegagalan, dan skalabilitas sistem IoT pertanian presisi.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam penelitian ini adalah:

1. Bagaimana merancang arsitektur sistem pemantauan dan kontrol aeroponik yang modular sehingga setiap komponen baik di sisi firmware maupun server dapat dikembangkan, diuji, dan di-*deploy* secara independen tanpa memengaruhi komponen lain?
2. Bagaimana menerapkan prinsip *database-per-service* dan komunikasi *event-driven* melalui NATS JetStream dan MQTT agar sistem mampu mengisolasi kegagalan antar-layanan dan menjamin konsistensi data secara terdistribusi?
3. Bagaimana mekanisme modularitas firmware ESP32 berbasis *factory pattern* dan *protocol registry* memungkinkan penambahan protokol baru (seperti protokol I2C sensor INA219) tanpa perlu mengubah logika inti firmware atau melakukan kompilasi ulang?
4. Bagaimana mengintegrasikan kapabilitas kecerdasan buatan berupa model kontrol berbasis *Reinforcement Learning* (TD3) dan deteksi visual berbasis YOLOv8 ke dalam arsitektur microservice tanpa menimbulkan *tight coupling* dengan layanan-layanan lainnya?
5. Sejauh mana sistem yang dibangun memenuhi kriteria kinerja (latensi, *throughput*, keandalan) yang diperlukan untuk operasional pemantauan aeroponik secara real-time?

1.3 Tujuan Penelitian

Penelitian ini bertujuan untuk:

1. **Merancang dan membangun** sistem pemantauan dan kontrol lingkungan aeroponik berbasis arsitektur microservice modular yang terdiri dari layanan-layanan backend mandiri, masing-masing dengan database, kontrak komunikasi, dan *deployment* tersendiri.

2. **Mengimplementasikan firmware modular** pada mikrokontroler ESP32 menggunakan pola *factorypattern* dan *protocolregistry* yang memungkinkan penambahan sensor dan protokol baru secara dinamis melalui konfigurasi tanpa kompilasi ulang.
3. **Membuktikan keunggulan modularitas** secara empiris melalui pengujian pada layanan-layanan inti (Auth Service, Module Service, Control Service, Analytics Service) serta pengujian penambahan handler protokol I2C untuk sensor INA219 pada firmware ESP32.
4. **Mengembangkan dan mengintegrasikan model kontrol berbasis *Reinforcement Learning* (TD3)** dari awal (*from scratch*) untuk kebutuhan integrasi kontrol penjadwalan penyemprotan adaptif.
5. **Mengevaluasi kinerja sistem** secara kuantitatif melalui pengujian unit, pengujian beban (*stress test*), dan pengujian resiliensi (*chaos engineering*), meliputi metrik latensi, *throughput*, dan tingkat keberhasilan pengiriman pesan.

1.4 Lingkup Permasalahan, Asumsi, dan Hipotesis

Agar cakupan penelitian tetap terarah dan terukur, batasan ruang lingkup masalah ditetapkan sebagai berikut:

1. Fokus penelitian berada pada perancangan, implementasi, dan pengujian modularitas arsitektur microservice serta integrasi kontrol adaptif aeroponik, bukan pada optimasi algoritma *Reinforce Learning*.
2. Pembahasan arsitektur server difokuskan secara mendalam pada **layanan-layanan inti**, yaitu: **Auth Service** (manajemen autentikasi terpusat, RBAC, dan penerbitan JWT), **Module Service** (registrasi perangkat dan *ingestion* telemetri), **Control Service** (arbitrase mode kendali dan eksekusi perintah aktuasi), serta **Analytics Service** (agregasi deret waktu dan komputasi metrik) sebagai representasi pembuktian modularitas, pemisahan database (*database-per-service*), dan komunikasi *event-driven*.

Layanan lainnya (Stream, Alert, Notification, Export, Audit, DLQ, WS-Gateway) berfungsi sebagai infrastruktur pendukung operasional.

3. Pengujian integrasi algoritma kontrol adaptif cerdas diteliti melalui alur terintegrasi yang melibatkan: **Auth Service**, **Module Service**, **Control Service**, **Stream Service**, **ML Service**, **model-control**, dan **model-controller**.
4. Algoritma kecerdasan buatan untuk visi komputer (YOLOv8) diambil langsung dari model pra-latih (*pre-trained / ready-to-use*) untuk mengekstraksi informasi visual (panjang akar dan kondisi umbi tanaman). Sementara itu, algoritma *Reinforcement Learning* (TD3) dikembangkan dari awal (*from scratch*) dengan lingkungan (*custom environment*) khusus aeroponik, namun ruang lingkupnya dibatasi hanya untuk kebutuhan integrasi kontrol penjadwalan adaptif, bukan sebagai studi optimasi matematis/algoritma RL secara teoritis.
5. Domain sensor yang dipantau dibatasi pada: suhu udara, kelembapan udara, suhu larutan nutrisi, *electrical conductivity* (EC), pH larutan, dan status aktuator (pompa misting dan katup nutrisi).
6. Firmware dikembangkan khusus untuk platform ESP32 dengan framework Arduino via PlatformIO di atas FreeRTOS dual-core; modularitas firmware diuji secara spesifik melalui skenario penambahan handler protokol I2C untuk sensor daya (seperti INA219) melalui pendaftaran konfigurasi config.json maupun melalui antarmuka lokal firmware.
7. Protokol komunikasi inter-service menggunakan NATS JetStream, komunikasi API (REST/HTTP) digunakan khusus untuk komunikasi dengan layanan luar/komputasi Python seperti **ML Service** dan **model-control / model-controller**, sedangkan komunikasi edge antara firmware ESP32 dan server menggunakan protokol MQTT via Mosquitto.
8. Antarmuka pengguna baik antarmuka lokal firmware (*Captive Web Portal*) maupun dashboard server (React Vite) dibatasi pembahasannya pada

pemenuhan fungsi-fungsi operasional monitoring, konfigurasi, dan kontrol data, tanpa membahas perancangan detail estetika antarmuka ataupun pengalaman pengguna (UI/UX).

Dalam pelaksanaan penelitian ini, digunakan asumsi-asumsi dasar sebagai berikut:

1. Jaringan WiFi lokal antara ESP32 dan server diasumsikan memiliki kualitas yang memadai untuk transmisi data telemetri dengan interval 5 detik secara stabil (latensi <500 ms, *packet loss* <1%).
2. Server *host* diasumsikan memiliki sumber daya komputasi minimal (RAM 8 GB, CPU 4 core, *No GPU*) yang memadai untuk menjalankan seluruh tumpukan Docker Compose secara bersamaan dalam lingkungan pengujian.
3. Sensor-sensor yang digunakan (suhu, EC, pH) diasumsikan telah dikalibrasi dan memberikan pembacaan yang akurat sesuai spesifikasi pabrikan sebelum pengujian dimulai.
4. Model TD3 yang dikembangkan diasumsikan telah dilatih pada lingkungan simulasi aeroponik dan siap untuk inferensi serta integrasi jadwal kontrol selama periode pengujian berlangsung.
5. Seluruh layanan backend diasumsikan dapat berkomunikasi melalui jaringan Docker internal tanpa hambatan firewall atau pembatasan *port* di lingkungan pengujian.

Hipotesis awal yang diajukan dalam penelitian ini adalah:

1. Penerapan pola *factory pattern* dan *protocol registry* pada firmware ESP32 memungkinkan penambahan handler protokol komunikasi baru (seperti I2C untuk sensor INA219) tanpa mengubah kode logika inti (*main loop* dan penjadwalan *task*) dan tanpa kompilasi ulang seluruh proyek.
2. Arsitektur *microservice* dengan prinsip *database-per-service* dan komunikasi *event-driven* via NATS JetStream pada layanan inti (Auth,

Module, Control, Analytics) mampu mengisolasi kegagalan satu layanan sehingga layanan lainnya tetap beroperasi secara normal.

3. Sistem mampu melayani penambahan jumlah node sensor tanpa penurunan kinerja yang signifikan latensi end-to-end tetap <2 detik pada persentil ke-95 karena skalabilitas beban dapat dilakukan secara selektif pada Module Service dan Analytics Service.
4. Integrasi model TD3 yang dikembangkan mampu mengeksekusi penyesuaian jadwal penyemprotan secara adaptif berdasarkan kombinasi telemetri dan deteksi visual, dengan stabilitas parameter lingkungan berada dalam rentang target selama minimal 80% periode pengujian.

1.5 Pendekatan, Metode, dan Pelaksanaan Penelitian

Penelitian ini menggunakan pendekatan **rekayasa sistem terapan** (*applied engineering research*) dengan kerangka kerja *Design Science Research* (DSR) [10]. Pelaksanaan penelitian secara garis besar dibagi menjadi empat fase utama, yaitu:

1. **Fase Perancangan Sistem:** Merancang arsitektur dua lapisan firmware ESP32 berbasis FreeRTOS dual-core dengan pola *factory pattern* dan *protocol registry* untuk manajemen sensor secara modular, serta topologi 12 layanan backend mandiri dengan prinsip *database-per-service* (8× MariaDB, 2× TimescaleDB, 1× Redis, 1× MinIO) yang terhubung melalui event bus NATS JetStream, REST API untuk service luar, dan API Gateway Kong.
2. **Fase Implementasi:** Pembangunan seluruh komponen sistem secara bertahap sesuai urutan prioritas dimulai dari infrastruktur (NATS, Mosquitto, Kong, database), layanan inti backend (Auth, Module, Analytics, Control), layanan pendukung (Alert, Notification, Stream, Audit, Export, DLQ), model AI (TD3 *control loop* dan YOLOv8 *vision pipeline*), fungsionalitas antarmuka (Captive Web Portal pada firmware dan

Dashboard React pada server), hingga observabilitas (Prometheus + Grafana) seluruhnya diorkestrasikan menggunakan Docker Compose.

3. **Fase Pengujian dan Verifikasi:** Meliputi pengujian unit per-layanan melalui Kong Gateway (/v1), pengujian beban (*stress test*) untuk mengukur *throughput* dan latensi di bawah beban simultan, pengujian resiliensi (*chaos engineering*) dengan mematikan layanan secara paksa untuk memverifikasi isolasi kegagalan, dan skenario penambahan handler protokol I2C untuk sensor DHT/BME pada firmware untuk membuktikan modularitas tanpa kompilasi ulang. Seluruh pengujian dijalankan menggunakan suite otomatis `test/run_all_tests.py` yang menghasilkan laporan visual dalam format PNG.
4. **Fase Evaluasi dan Analisis:** Menganalisis hasil pengujian secara kuantitatif menggunakan metrik latensi end-to-end (target <2 detik pada persentil ke-95), tingkat keberhasilan pengiriman pesan, jumlah layanan yang tetap beroperasi saat terjadi kegagalan, serta jumlah komponen yang perlu dimodifikasi saat menambahkan fitur baru. Hasil analisis ini digunakan untuk memverifikasi keempat hipotesis. Tahapan pengerjaan tersebut disajikan secara rinci pada Gambar 1.1 yang memuat diagram alir penelitian.

1.6 Sistematika Penulisan

Sistematika penyusunan laporan tugas akhir ini dibagi menjadi lima bab utama.

1. Bab I menjabarkan latar belakang masalah (pertanian presisi, arsitektur microservice modular, dan skalabilitas IoT), perumusan masalah, tujuan penelitian, batasan lingkup dan asumsi hipotesis, hingga metodologi rekayasa sistem.
2. Bab II memaparkan tinjauan pustaka yang mencakup teori dasar sistem aeroponik, prinsip arsitektur microservice (*database-per-service*, API Gateway, event-driven), protokol komunikasi IoT (MQTT dan NATS

JetStream), serta landasan model *reinforcement learning* (TD3) dan visi komputer (YOLOv8).

3. Bab III menjelaskan prosedur metode penelitian dan perancangan sistem yang mencakup: arsitektur sistem menyeluruh, perancangan firmware ESP32 dan modularitasnya (FreeRTOS, *factory pattern*, *protocol registry*), perancangan layanan-layanan inti backend (Auth, Module, Control, Analytics), perancangan model kontrol adaptif TD3 dan integrasi deteksi visual YOLOv8, integrasi layanan-layanan tambahan (Stream, ML, model-control, model-controller, Alert, dll.) sebagai pembuktian modularitas, serta perancangan fungsionalitas antarmuka pengguna pada firmware (*Captive Web Portal*) dan server (*React Dashboard*).
4. Bab IV menguraikan hasil implementasi sistem, proses pengujian fungsional dan performa (unit test layanan inti, *stress test*, *chaos engineering*, pembuktian modularitas firmware via protokol I2C untuk sensor INA219, dan pengujian alur integrasi algoritma kontrol), analisis metrik kinerja (latensi, throughput, isolasi kegagalan), serta pembahasan perbandingan terhadap hipotesis yang diajukan.
5. Bab V berisi rincian kesimpulan akhir dari penelitian termasuk pembuktian empiris empat hipotesis keterbatasan sistem yang teridentifikasi, serta implikasi praktis dan rekomendasi untuk kajian dan pengembangan lanjutan.

BAB II

DASAR TEORI

Pastikan anda sudah menempatkan Heading 1 pada setiap bab. Bab tinjauan pustaka berisi uraian tentang *state of the art* dari penelitian dan posisi yang diperoleh kandidat doktor. Pada hakikatnya, hasil penelitian seorang peneliti bukanlah satu penemuan baru yang berdiri sendiri melainkan sesuatu yang berkaitan dengan hasil penelitian sebelumnya.

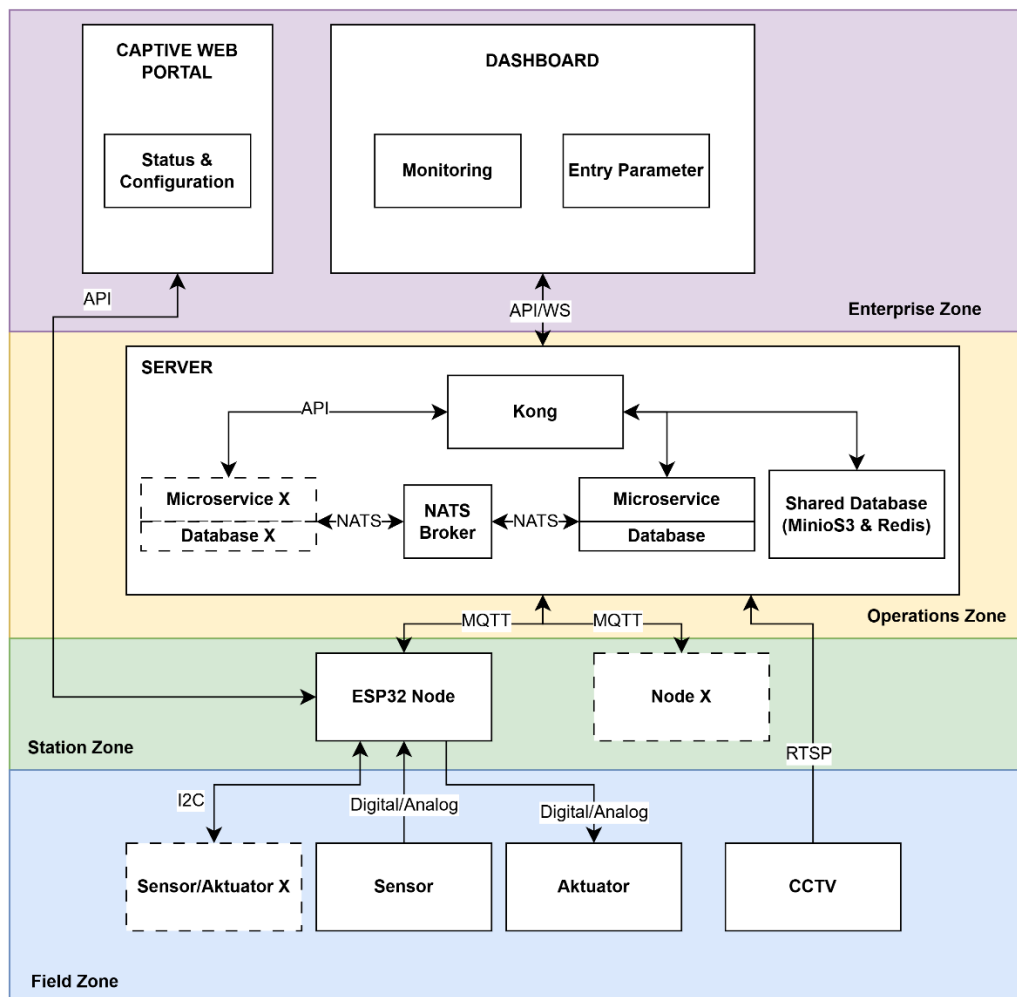
Dasar teori hendaklah disusun sesuai dengan urutan perkembangan cabang ilmu pengetahuan yang dikandungnya. Tinjauan pustaka berisi pula ulasan tentang kesimpulan yang terdapat dalam setiap judul dalam daftar pustaka dan dalam hubungan ini kandidat doktor menunjukkan mengapa dan bagaimana dipilihnya masalah penelitian serta arah yang akan ditempuhnya dalam menyelesaikan masalah penelitiannya.

BAB III

PERANCANGAN SISTEM DAN METODOLOGI

Bab ini menguraikan metodologi dan tahapan perancangan sistem dengan menerapkan paradigma modularitas ganda (*dual-layer modularity*), yang terbagi atas modularitas *firmware* pada perangkat tepi (*edge*) dan modularitas arsitektur *server*. Penerapan modularitas pada kedua lapisan ini ditekankan untuk memberikan kemudahan, baik dari aspek pengembangan (*development*) maupun penggunaan (*usability*) sistem secara berkelanjutan. Sebagai penutup, bab ini menyajikan penjabaran skenario pengujian komprehensif untuk memvalidasi performa serta keandalan dari sistem yang dirancang.

3.1 Arsitektur Sistem

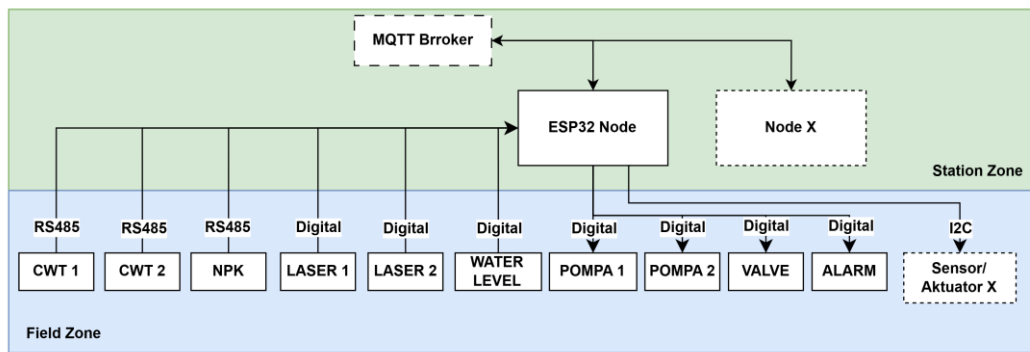


Gambar 3.1 Gambaran umum arsitektur sistem

Sistem ini dirancang dengan arsitektur terdistribusi yang terbagi menjadi empat zona fungsional sebagai berikut:

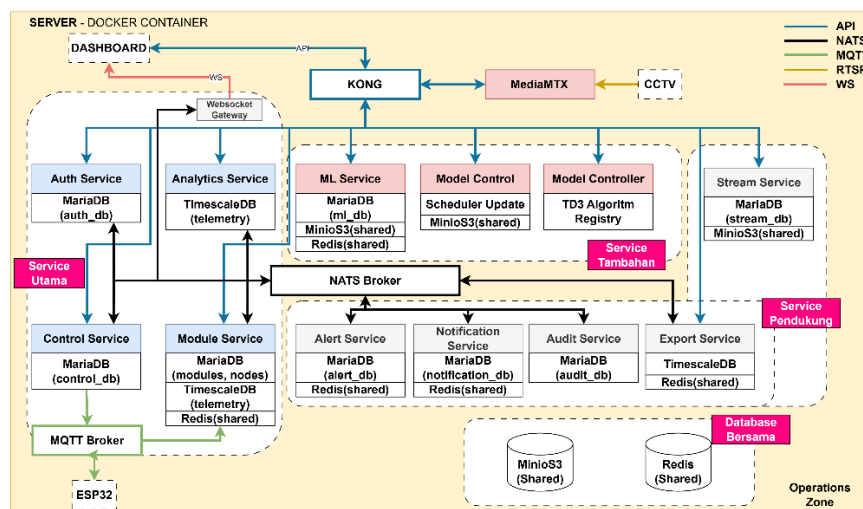
1. **Field Zone:** Bertujuan untuk memantau dan memanipulasi kondisi lingkungan fisik, yaitu *module* aeroponik, dengan memanfaatkan berbagai sensor pengukur dan aktuator.
2. **Station Zone:** Berfungsi sebagai perantara komputasi tepi (*edge*) untuk akuisisi data dari sensor di lapangan dan mendistribusikan perintah aktuasi ke perangkat aktuator.
3. **Operations Zone:** Bertugas sebagai pusat komputasi yang menjalankan seluruh logika operasi sistem. Zona ini dibangun di atas arsitektur *microservice*, di mana setiap layanan memiliki sebuah tugas spesifik.
4. **Enterprise Zone:** Berperan sebagai antarmuka (*interface*) utama bagi pengguna akhir untuk berinteraksi, mengonfigurasi, dan memantau sistem.

Untuk memastikan aliran data yang efisien di seluruh zona, sistem ini menggunakan beberapa protokol komunikasi yang disesuaikan dengan kebutuhan setiap lapisan. Untuk komunikasi antara sensor/aktuator dengan edge menggunakan sinyal digital dan analog serta protokol komunikasi I2C. Kemudian MQTT digunakan sebagai satu - satunya protokol komunikasi antara edge dan server. Sedangkan komunikasi internal antar *microservice* menggunakan NATS Jetstream. REST API digunakan untuk komunikasi eksternal dan websocket sebagai komunikasi eksternal realtime. RTSP sebagai protokol komunikasi cctv dengan server.



Gambar 3.2 Konfigurasi sensor/aktuator

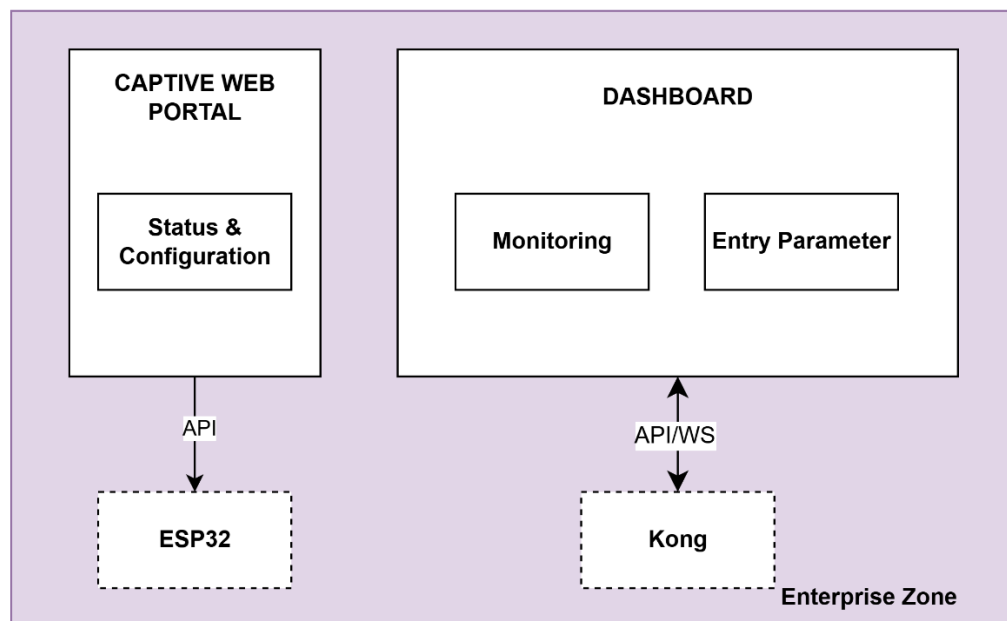
Konfigurasi perangkat keras pada sistem aeroponik ini mengintegrasikan **enam unit sensor**—mencakup sensor CWT, NPK, laser, dan *water level*—serta **empat unit aktuator** yang terdiri atas dua pompa, satu katup (*valve*), dan satu alarm. Fokus utama pengembangan terletak pada aspek modularitas perangkat keras yang mempermudah penambahan sensor maupun aktuator baru secara dinamis melalui penerapan pola desain *Protocol Registry* dan *Factory Pattern* pada lapisan *firmware*. Melalui pendekatan ini, pengembang cukup membangun protokol komunikasi baru (seperti protokol I2C) beserta *handler* terkait yang mengimplementasikan standar komunikasi sensor tersebut; selanjutnya, pengguna akhir dapat langsung mengintegrasikan perangkat keras baru secara deklaratif hanya dengan memasukkan jenis dan alamat sensor melalui berkas `config.json` atau antarmuka pengguna tanpa memerlukan proses kompilasi ulang (*recompilation*).



Gambar 3.3 Arsitektur microservice

Di sisi server, sistem dibangun dengan pendekatan arsitektur microservice yang berjalan didalam container Docker, setiap layanan hanya melayani satu jenis transaksi yang spesifik (*single responsibility*). Terdapat 13 layanan microservice dengan 12 *service* yang menerapkan prinsip *database-per-service*, dengan total 12 *instance* database terpisah yang terdiri atas 8× MariaDB, 2× TimescaleDB, 1× Redis, dan 1× MinIO. Seluruh layanan terhubung melalui *event bus* NATS JetStream, REST API digunakan untuk komunikasi antar layanan eksternal, dan Websocket Gateway untuk komunikasi real-time, serta satu-satunya pintu masuk dari luar yaitu Kong Gateway. Rincian masing-masing layanan dijelaskan pada Lampiran A.1.

Dari ke-13 layanan tersebut, ML Service, model-control, dan model-controller dikembangkan sebagai layanan eksternal yang berkomunikasi dengan layanan inti melalui REST API. Ketiga layanan ini bertujuan mengendalikan aktuatur pompa misting dan valve secara adaptif.



Gambar 3.4 Lapisan presentasi

Captive Web Portal dan *Dashboard* menempati lapisan presentasi pada *Enterprise Zone*. *Captive Web Portal* berjalan secara lokal pada perangkat ESP32 untuk konfigurasi di tingkat *edge*, sedangkan *Dashboard* sebagai monitoring dan entry

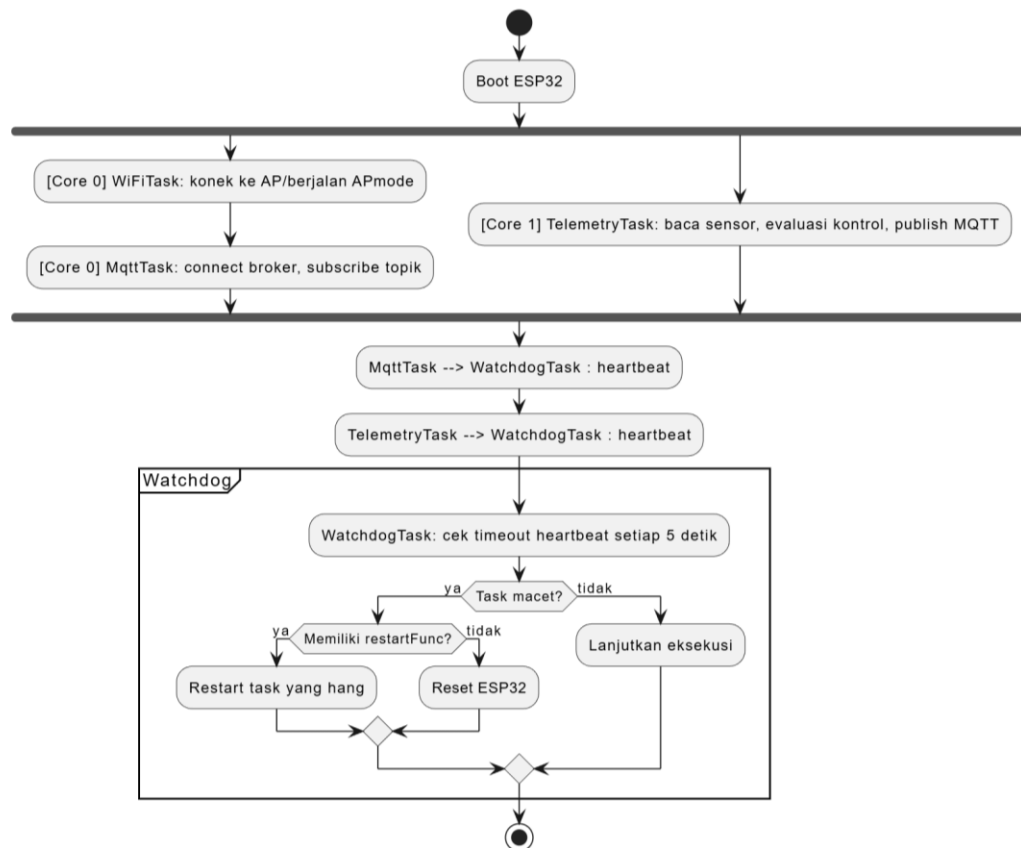
parameter yang berjalan di sisi server serta keduanya dapat diakses melalui *browser*.

3.2 Perancangan Firmware ESP32

Firmware ESP32 dirancang diatas FreeRTOS dengan pendekatan modular configuration-driven dan factory pattern. Dimana konfigurasi ini memungkinkan sensor atau aktuator baru cukup didaftarkan via file konfigurasi, tanpa mengubah inti program.

3.2.1 FreeRTOS

Desain ini memanfaatkan kinerja *dual-core* ESP32 yang bertujuan untuk membagi beban komputasi secara efisien dan memastikan keandalan eksekusi tugas fisik maupun komunikasi tanpa adanya pemblokiran (*non-blocking*).

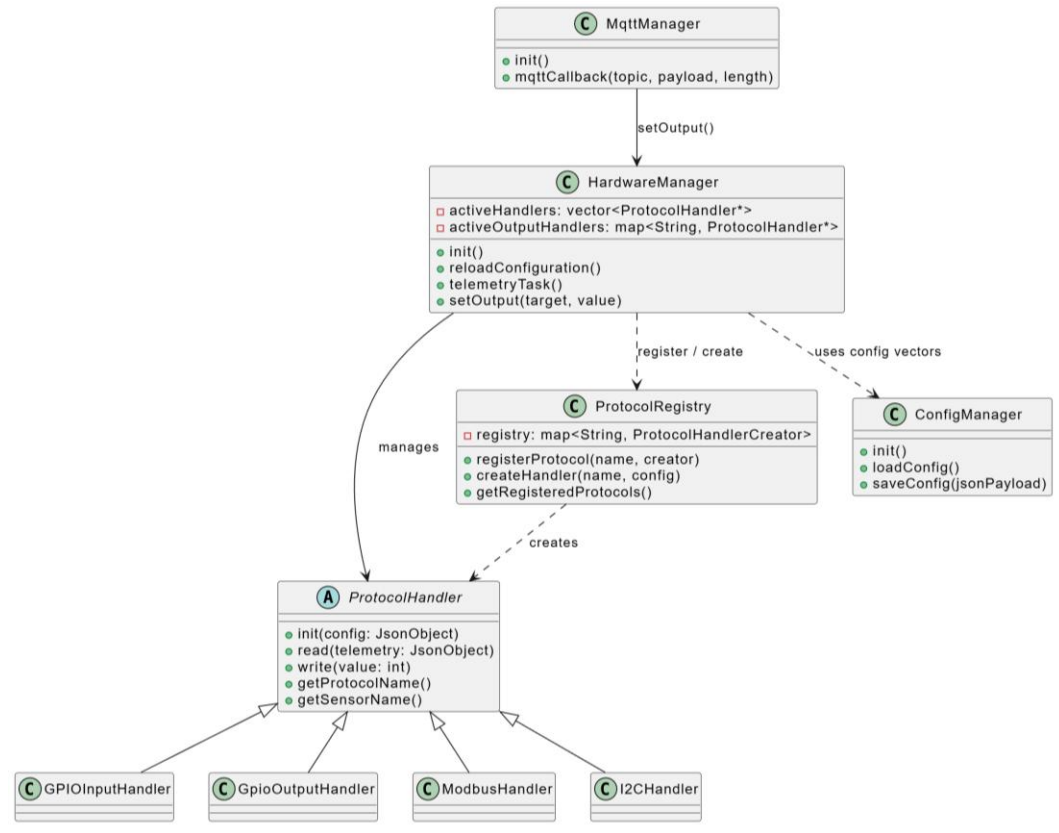


Gambar 3.5 Activity Diagram - Core Runtime ESP32 FreeRTOS

Core 0 menangani semua komponen jaringan dan pengawasan sistem, yaitu WiFiTask untuk menjaga koneksi internet dan melayani Captive Portal, MqttTask untuk mengelola komunikasi dengan broker MQTT dan menerima perintah aktuator, serta WatchdogTask yang berfungsi sebagai pengawas kesehatan seluruh task. SysMonitorTask untuk memantau penggunaan memori dan DiscoveryPeriodic yang secara otomatis dibuat oleh MqttTask untuk mengirimkan pesan discovery setiap 60 detik guna memastikan module service selalu mengetahui node yang aktif.

Core 1 dialokasikan untuk TelemetryTask yang bertugas membaca sensor secara periodik setiap 5 detik (default), memproses data GPIO, Modbus RS485, dan I2C, lalu memformatnya menjadi JSON sebelum diteruskan ke MqttTask untuk dipublikasikan. Hanya MqttTask dan TelemetryTask yang mengirim heartbeat ke WatchdogTask. Jika salah satu task berhenti mengirim heartbeat, watchdog akan me-restart task tersebut atau memicu reset sistem, sehingga kegagalan satu komponen tidak memblokir operasi lain.

3.2.2 Modular I/O : Factory Pattern & Registry



Gambar 3.6 Class Diagram - Factory Pattern & Registry ProtocolHandler

ConfigManager memuat config.json dari LittleFS saat boot dan memetakan elemen JSON ke dalam `std::vector` in-memory di namespace Config: HardwareInputs, HardwareOutputs, HardwareModbus, HardwareSensors, dan LocalControlRules. Di config.json, protokol didefinisikan hanya sebagai string seperti "GPIO", "MODBUS", "I2C", atau "GPIO_OUT".

Saat `HardwareManager::init()` dijalankan, semua handler didaftarkan ke ProtocolRegistry sebagai peta `String → lambda creator`. Setiap kali konfigurasi diubah melalui Captive Portal, `reloadConfiguration()` memanggil `ProtocolRegistry::createHandler()` untuk membuat instance handler sesuai nama protokol di JSON, lalu menyimpannya ke `activeHandlers` (sensor) atau `activeOutputHandlers` (aktuator).

TelemetryTask mengiterasi `activeHandlers` dan memanggil `read()` untuk setiap sensor, sedangkan `MqttCallback` menerima perintah aktuator dan memanggil

HardwareManager::setOutput() yang pada akhirnya memanggil write() pada handler aktuator yang sesuai.

3.2.3 Vector Registry

Vector Registry merupakan mekanisme penyimpanan konfigurasi sensor ke dalam sebuah *list* (vektor) yang tersusun berurutan. Pendekatan ini dipilih karena TelemetryTask membaca seluruh sensor secara berkala dalam satu siklus. Dengan vektor, proses pembacaan cukup dilakukan dengan iterasi sekuensial firmware berjalan dari elemen pertama hingga terakhir tanpa perlu melakukan pencarian berdasarkan nama sensor (*lookup by name*). Dengan menggunakan metode ini pembacaan sensor dapat berjalan lebih efisien dan memiliki urutan telemetry yang konsisten.

```
// HardwareManager.cpp – deklarasi
std::vector<ProtocolHandler*> activeHandlers;
```

Kode diatas adalah implementasi dari vector registry untuk activeHandlers yang merupakan array dinamis untuk menampung pointer ke objek ProtokolHandler. Karena tipenya ProtocolHandler* (pointer ke *base class*), vektor ini dapat menyimpan handler dari berbagai jenis sekaligus GPIOInputHandler, ModbusHandler, I2CHandler berkat sifat *polymorphism*. Jadi nantinya activeHandlers akan berisi semua sensor yang aktif, apapun protokol komunikasinya.

Selanjutnya adalah membangun handler yang akan menangani protokol komunikasi langsung ke sensor. Seluruh kode program handler dapat dilihat pada Lampiran B.9 -B.12.

Fungsi GPIOInputHandler::read menangani akuisisi data sensor dari pin GPIO, baik bertipe analog maupun digital. Alur kerja dimulai dengan menginisialisasi sub-objek "inputs" pada struktur JSON telemetry. Bergantung pada tipe konfigurasinya, program akan melakukan pembacaan analog mentah (ADC 12-bit) atau pembacaan digital biner dengan opsi inversi logika (*active-low*). Hasil akhir pembacaan kemudian dimasukkan ke dalam *payload* JSON telemetry untuk dikirim ke server, sekaligus disimpan ke dalam *local cache* (latestSensorValues) guna mendukung

keputusan kendali lokal secara waktu nyata (*real-time*). Data yang telah diproses dipetakan ke dalam objek inputs menggunakan nama sensor (name) sebagai kunci (key).

```
{
  "inputs": {
    "soil_moisture": 2048
  }
}
```

Fungsi `ModbusHandler::read` yang menangani akuisisi data sensor Modbus RTU melalui antarmuka serial RS485 secara aman dan terisolasi. Alur kerja dimulai dengan mengamankan bus serial menggunakan mekanisme saling-pengecualian (*mutex*) `modbusMutex` demi mencegah tabrakan data (*collision*), dilanjutkan dengan penyesuaian kecepatan transmisi (*baud rate*) secara dinamis. Melalui iterasi daftar konfigurasi, fungsi ini membaca *Input Register* atau *Holding Register* sesuai parameter target, lalu mengalikan nilai mentah hasil pembacaan dengan faktor pengali (*multiplier*). Hasil akhirnya kemudian dipetakan secara terstruktur ke dalam sub-objek "modbus" pada *payload* JSON telemetri sekaligus disimpan pada *local cache* (`latestSensorValues`) untuk kebutuhan kendali lokal secara waktu nyata (*real-time*).

Sebagai ilustrasi, apabila sensor dengan parameter nama perangkat "soil_sensor" mengidentifikasi register kelembapan "humidity" dengan nilai terkalibrasi 45.5, maka struktur luaran JSON konseptual yang dihasilkan adalah sebagai berikut:

```
{
  "modbus": {
    "soil_sensor": {
      "humidity": 45.5
    }
  }
}
```

Fungsi `I2CHandler::read` pada sensor bertipe INA219 menangani akuisisi parameter kelistrikan sistem melalui bus komunikasi I2C. Alur kerja dimulai dengan menginisialisasi sub-objek nama perangkat (seperti "power_monitor") di dalam objek "i2c" pada JSON telemetri. Program kemudian mengukur nilai

tegangan bus (*bus voltage*), tegangan *shunt*, arus beban (*current*), dan konsumsi daya (*power*) secara langsung dari modul sensor. Parameter tersebut dipetakan ke dalam *payload* JSON telemetri sekaligus disimpan pada *local cache* (`latestSensorValues`) guna mendukung kebutuhan pemantauan dan manajemen daya secara waktu nyata (*real-time*).

Sebagai ilustrasi, berdasarkan pembacaan sensor INA219 dengan nama perangkat "power_monitor", struktur luaran JSON konseptual yang dihasilkan adalah sebagai berikut:

```
{
  "i2c": {
    "power_monitor": {
      "bus_voltage_v": 5.02,
      "shunt_voltage_mv": 14.26,
      "current_ma": 142.6,
      "power_mw": 716.0
    }
  }
}
```

3.2.4 Map Registry

```
// HardwareManager.cpp - deklarasi
std::map<String, ProtocolHandler*> activeOutputHandlers;
```

Berbeda dengan komponen sensor yang disimpan menggunakan struktur `std::vector` untuk kebutuhan iterasi secara massal (*batch*), modul aktuator menerapkan struktur peta pencarian (*map*) `activeOutputHandlers`. Pemilihan struktur *map* ini didasarkan pada kebutuhan pencarian leksikal cepat dengan kompleksitas algoritma $O(\log n)$ tanpa melalui proses perulangan (*looping*) saat fungsi `setOutput()` menerima parameter identitas aktuator. Sama halnya dengan sensor, aktuator juga mengimplementasikan abstraksi dari kelas induk `ProtocolHandler` dan mendaftarkan kelas konkretnya pada `ProtocolRegistry`, namun dengan penambahan kontrak metode virtual murni `write(int value)` untuk menangani instruksi penulisan status fisik perangkat secara terisolasi.

Ketika `MqttManager` menerima pesan instruksi dari server melalui topik `smartfarm/actuator/<node_id>`. Sebagai contoh, ketika server mengirimkan pesan perintah berupa muatan JSON berikut:

```
{
  "action": "set_output",
```

```
"target": "load1",  
"value": 0  
}
```

Fungsi pendengar mqttCallback segera merespons dengan mengurai (parsing) muatan JSON tersebut menjadi tiga parameter operasional utama, yaitu tindakan (action = "set_output"), target aktuator (target = "load1"), dan nilai keluaran (value = 0).

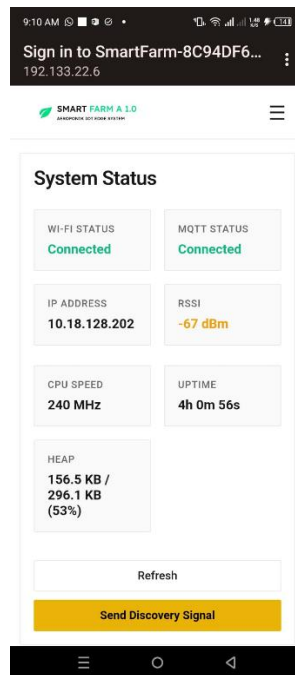
Ketiga parameter ini kemudian diteruskan ke fungsi setOutput(targetName, value) kode tersedia pada Lampiran B.14 sebagai jembatan logika internal. Jika target "load1" ditemukan di dalam peta registrasi activeOutputHandlers, fungsi akan mengeksekusi metode write(0), memperbarui status operasional terakhir pada registri outputStates, serta mengirimkan sinyal interupsi RTOS melalui instruksi xTaskNotifyGive(telemetryTaskHandle) guna memicu tugas TelemetryTask agar segera mengirimkan laporan pembaruan status aktuator ke server secara instan.

Kelas GpioOutputHandler::write kode program tersedia pada Lampiran B.15 mengeksekusi perintah berdasarkan jenis konfigurasi pin aktuator. Apabila dikonfigurasi sebagai PWM (Pulse Width Modulation), sistem mengeksekusi perintah analogWrite dengan rentang value 0 sampai 255. Sebaliknya, jika dikonfigurasi sebagai DIGITAL, sistem mengeksekusi perintah digitalWrite (HIGH/LOW) untuk mengubah status kelistrikan pin GPIO secara langsung.

Sebagai ilustrasi, setelah perintah penonaktifan beban "load1" berhasil dieksekusi secara fisik melalui firmware mengirim pesan konfirmasi ke topik smartfarm/node-01/confirm yang dikirim ke server seperti berikut:

```
{  
  "req_id": "550e8400-e29b-41d4-a716-446655440000",  
  "target": "load1",  
  "value": 0,  
  "status": "executed"  
}
```

3.3 Antarmuka Firmware: Captive Web Portal



Gambar 3.7 Captive Web Portal

Untuk mempermudah pengelolaan konfigurasi berkas `config.json`, sistem menyediakan antarmuka web responsif yang dapat diakses melalui mekanisme *Captive Web Portal*. Akses antarmuka dilakukan dengan menghubungkan perangkat pengguna ke titik akses nirkabel (*wireless access point*) yang dipancarkan oleh ESP32 dengan SSID SmartFarm-<node_id>, lalu mengakses alamat IP instansi 192.168.133.6 melalui peramban (*browser*). Setelah berhasil melakukan autentikasi menggunakan kredensial bawaan (*default*: admin/admin1234), pengguna akan diarahkan ke halaman dasbor utama.

3.4 Standarisasi Komunikasi MQTT

MQTT beroperasi sebagai satu-satunya jalur komunikasi yang menjembatani *Station Zone* (node) dan *Operations Zone* (server). Oleh karena itu standarisasi komunikasi MQTT diperlukan untuk efisiensi transfer data. Selain itu, standarisasi ini juga memastikan kemudahan integrasi dan pengembangan sistem di masa depan, sehingga berbagai jenis node yang berbeda dapat tetap terhubung ke server menggunakan protokol yang sama.

3.4.1 Parameter dan Kontrak Topik

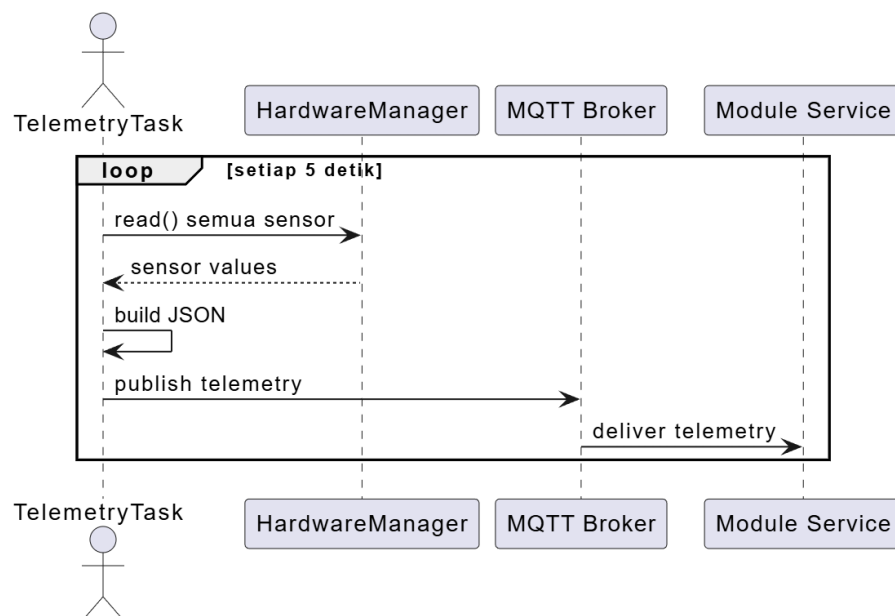
Parameter koneksi MQTT (seperti broker host, port, dan topic prefix) tersimpan secara dinamis dan dapat dikonfigurasi via Captive Web Portal. Struktur topik MQTT menerapkan pola hierarki:

```
{prefix}/{domain}/{node_id}

#{prefix} = smartfarm (configurable via MQTT_TOPIC_PREFIX)
#{domain} = kategori pesan (telemetry, actuator, confirm, status, discovery, dll.)
#{node_id} = identitas unik perangkat (misalnya: node-01, node-02)
```

Pola ini memungkinkan penggunaan wildcard subscription di sisi server, seperti smartfarm/+/telemetry untuk memantau seluruh telemetry atau smartfarm/actuator/+ untuk seluruh perintah kontrol. Detail lengkap mengenai kontrak topik terdapat pada Lampiran C.1.

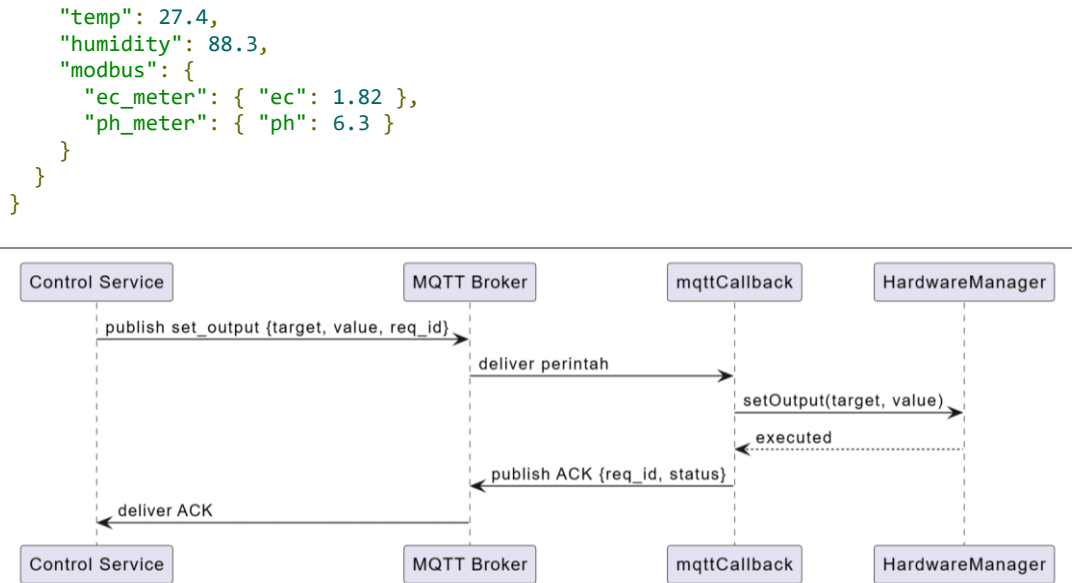
3.4.2 Standar Output Telemetry dan Kendali Aktuator



Gambar 3.8 Sequence Diagram — Telemetry Flow

Firmware mengirimkan data (smartfarm/{node_id}/telemetry) sensor fisik, metrik jaringan, dan status hardware ke Module Service (default 5 detik) dengan struktur JSON:

```
{
  "node_id": "node-01",
  "telemetry": {
    "outputs": { "pump": 0, "valve": 1 },
```



Gambar 3.9 Sequence Diagram — Actuator Command Flow

Control Service mengirim perintah ke topik smartfarm/actuator/{node_id} dengan struktur:

```

{
  "action": "set_output",
  "target": "pump",
  "value": 1,
  "req_id": "cmd-uuid-f4a7c2e1"
}

```

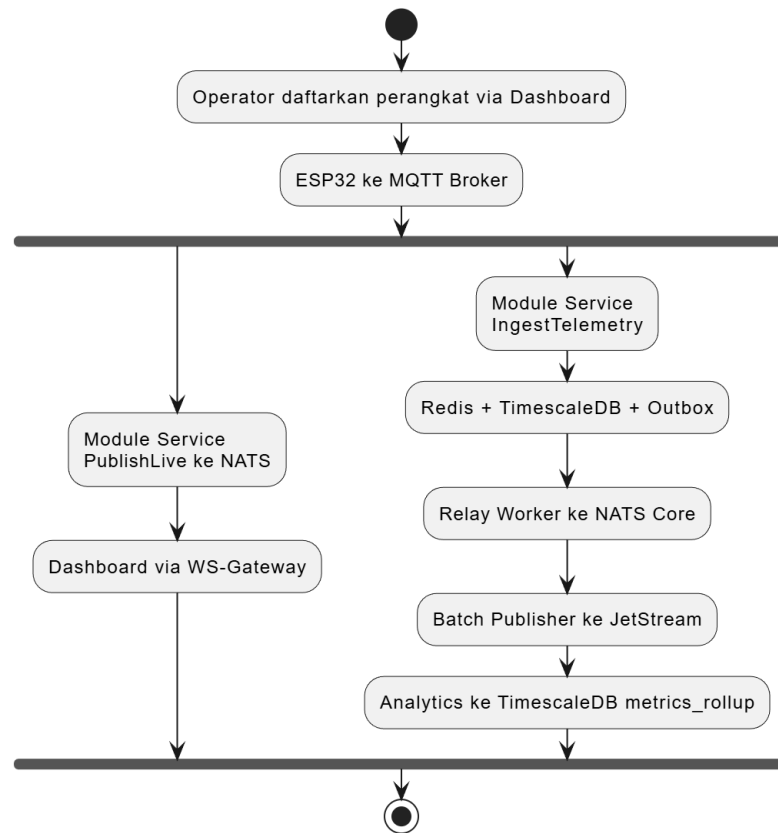
Kemudian perintah diterima oleh mqttCallback, diparse, kemudian diteruskan ke HardwareManager::setOutput() yang mencari handler aktuator di activeOutputHandlers berdasarkan nama target dan memanggil write(value). Setelah eksekusi berhasil, ESP32 mengirim ACK ke smartfarm/{node_id}/confirm:

```

{
  "req_id": "cmd-uuid-f4a7c2e1",
  "target": "pump",
  "value": 1,
  "status": "executed"
}

```

3.5 Ingesti Data Telemetry Module Service



Gambar 3.10 Data Flow Diagram — Telemetry Pipeline

Module Service bertanggung jawab mengelola komunikasi MQTT langsung dengan ESP32 melalui akuisisi data telemetry dan status perangkat, melakukan manajemen modul serta registrasi node otomatis via sinyal discovery, menyimpan data time-series secara efisien di TimescaleDB, serta meneruskan seluruh *event* ke layanan *microservice* lain melalui NATS JetStream.

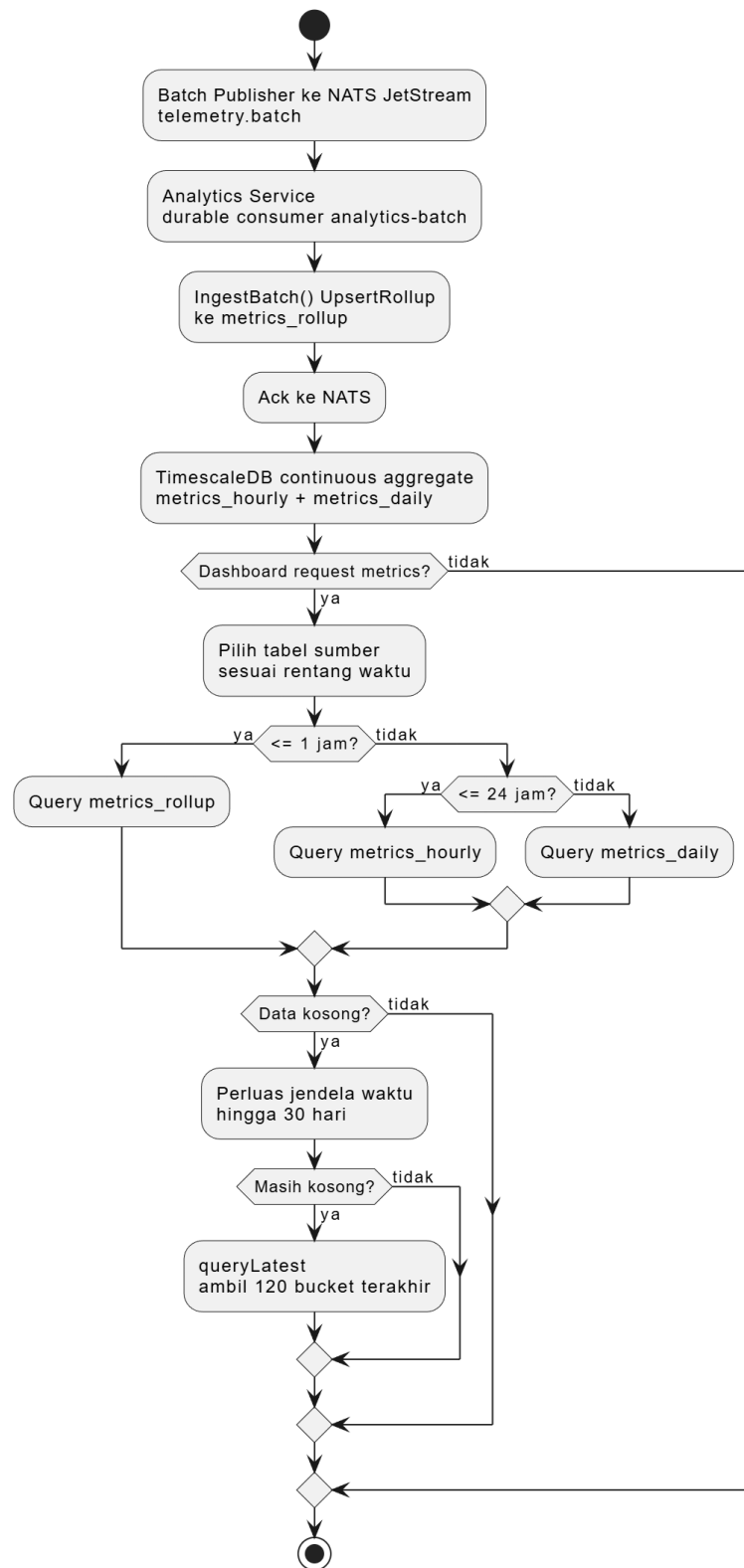
Sebelum data telemetry dapat diolah oleh Module Service, perangkat perlu didaftarkan terlebih dahulu oleh operator melalui *dashboard* konfigurasi dapat dilihat Lampiran D.1. Setelah ESP32 mempublikasikan data telemetry ke MQTT Broker, data tersebut diterima oleh fungsi `onMessage()` pada komponen `subscriber.go`.

Fungsi `onMessage()` kemudian membagi alur pemrosesan data menjadi dua jalur:

1. Jalur Real-time (PublishLive()): Payload mentah dari ESP32 langsung diteruskan ke NATS (mqtt.{node_id}) untuk ditampilkan secara *real-time* pada *dashboard* pengguna melalui WebSocket-Gateway.
2. Jalur Ingest & Penyimpanan (IngestTelemetry()):
 - Data telemetry terlebih dahulu disimpan di Redis *cache* untuk pengaksesan cepat.
 - Selanjutnya, fungsi IngestTelemetry() melakukan ekstraksi nilai metrik dari payload JSON bersarang menggunakan aturan pemetaan (*tag mapping*) yang tersimpan pada *in-memory cache*. Pendekatan ini meniadakan *query* metadata ke MariaDB pada setiap pengiriman data sehingga proses akuisisi berjalan sangat cepat.
 - Nilai metrik numerik yang berhasil dipetakan kemudian ditulis langsung ke *hypertable* telemetry pada TimescaleDB sebagai data historis *time-series*.
 - Untuk menginformasikan kedatangan data ke layanan *backend* lainnya, sistem mengantrekan *event* dengan menuliskan catatan kejadian ke tabel *outbox* MariaDB dalam transaksi lokal.

Setelah proses ingest selesai, komponen **Relay Worker** mem-poll tabel *outbox* secara berkala untuk menerbitkan pesan ke NATS Core dengan *header* deduplikasi, menjamin prinsip *at-least-once delivery* meskipun *message broker* sempat terganggu. Di saat yang sama, **Batch Publisher** meng-agregasi telemetry per 1 menit ke NATS JetStream, yang selanjutnya dikonsumsi oleh **Analytics Service** untuk menyimpan hasil agregasi ke dalam tabel *metrics_rollup* di TimescaleDB.

3.6 Pemrosesan Data Analytics Service



Gambar 3.11 Data Flow Diagram — Analytics Pipeline

Pesan telemetry `telemetry.batch` yang diterbitkan melalui NATS JetStream dikonsumsi oleh Analytics Service sebagai *durable consumer* `analytics-batch` menggunakan metode `DeliverAll()` dan `ManualAck`. Proses konsumsi ini berjalan secara independen dan asinkron, di mana setiap pesan diolah melalui fungsi `IngestBatch()` dan di-*upsert* langsung ke tabel `metrics_rollup` pada database TimescaleDB (`timescaledb-analytics`).

Data disimpan sementara oleh NATS dengan retensi 24 jam (`FileStorage`). Jika Analytics Service mengalami *down* atau *restart*, data yang belum terproses akan di-*replay* otomatis saat layanan kembali aktif. Sinyal pengakuan (*Ack*) hanya dikirim ke NATS setelah penyimpanan ke database berhasil. Jika terjadi kesalahan teknis saat menyimpan data, NATS akan mendistribusikan ulang (*redeliver*) pesan tersebut.

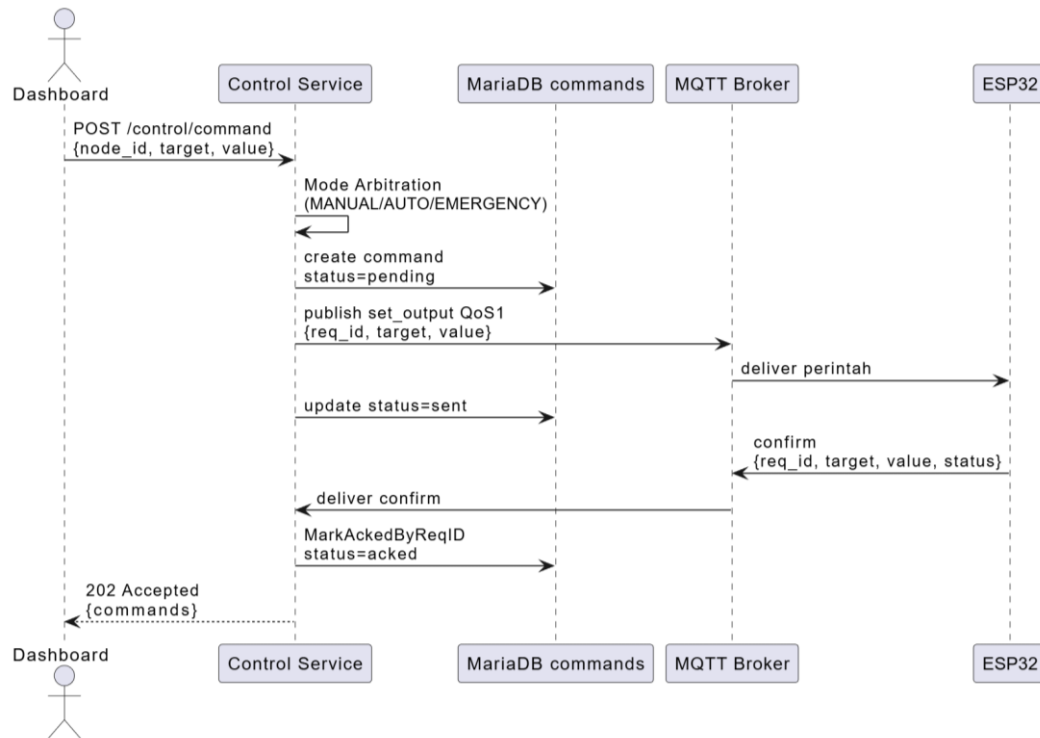
Untuk menghemat memori dan mempercepat waktu *query*, TimescaleDB secara otomatis mengagregasi data 1-menit dari `metrics_rollup` ke tabel agregasi bertingkat, yaitu `metrics_hourly` dan `metrics_daily`, melalui fitur *continuous aggregate*. Penggunaan tabel agregasi bertingkat ini bertujuan untuk memangkas ukuran *payload* dan mempercepat proses *rendering* grafik pada *dashboard* tanpa harus memproses ribuan titik data mentah.

Saat *dashboard* meminta data grafik (`GET /v1/analytics/metrics`), sistem secara otomatis memilih tabel sumber data berdasarkan rentang waktu yang diminta:

- `metrics_rollup` untuk rentang waktu ≤ 1 jam,
- `metrics_hourly` untuk rentang waktu ≤ 24 jam, dan
- `metrics_daily` untuk rentang waktu > 24 jam.

Jika tidak ada data pada rentang waktu yang diminta, sistem memperluas jendela pencarian secara bertingkat hingga 30 hari. Apabila node sudah lama nonaktif dan data masih kosong, sistem menjalankan fungsi `queryLatest` untuk mengambil 120 *bucket* terakhir dari `metrics_rollup`. Mekanisme ini menjamin *dashboard* selalu menampilkan grafik atau titik data terakhir, bukannya grafik kosong.

3.7 Manajemen Kontrol Pada Control Service



Gambar 3.12 Sequence Diagram — Control Service Command

Control Service bertindak sebagai *control plane* backend utama yang bertanggung jawab mengelola seluruh siklus hidup perintah aktuator. Layanan ini menghubungkan antarmuka pengguna (*dashboard*) dan eksekusi otomatis (*scheduler internal*) dengan *firmware* ESP32 secara aman, non-blocking, serta terstruktur.

Seluruh instruksi eksekusi bermuara pada satu fungsi utama (`Dispatch()`) untuk divalidasi oleh komponen **Mode Arbitration** guna mencegah bentrok (*race condition*) antara perintah manual pengguna dan otomatisasi *scheduler*:

- **MANUAL:** Memprioritaskan intervensi pengguna dari *dashboard*. Seluruh sinyal perintah dari *scheduler* internal akan langsung diabaikan.
- **AUTO:** *Scheduler* memegang kendali penuh atas sistem. Perintah manual dari *dashboard* akan ditolak secara otomatis, kecuali jika membawa parameter *bypass* khusus (seperti dari modul *Reinforcement Learning* / layanan eksternal).

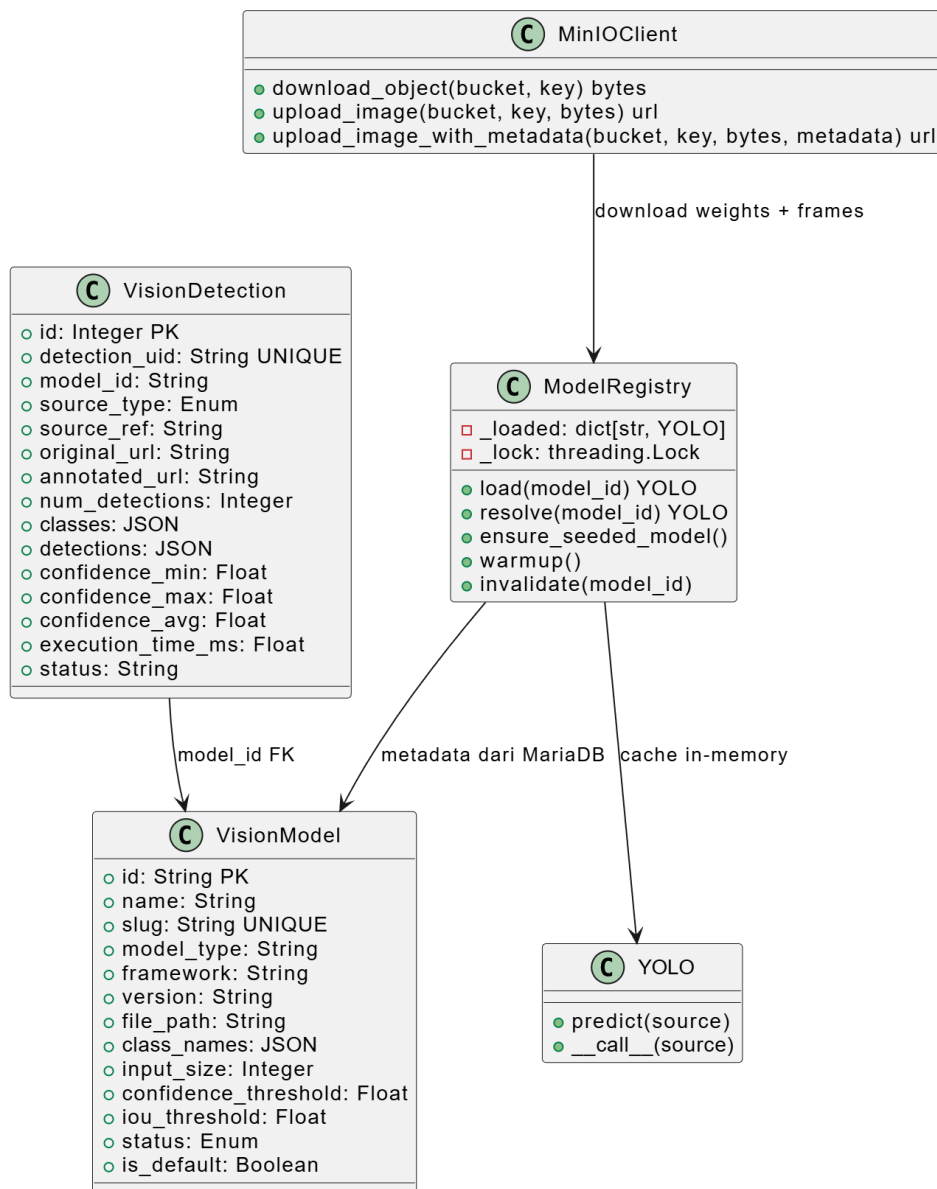
- **EMERGENCY:** Mengunci seluruh jalur eksekusi dan memaksa status aktuator ke kondisi mati (**OFF**). Mode operasional sebelum darurat disimpan ke dalam variabel *prevMode* agar sistem dapat kembali ke kondisi normal (*resume*) secara aman.

Pengiriman perintah ke ESP32 menggunakan topik MQTT *smartfarm/actuator/{node_id}* dengan jaminan *Quality of Service* (QoS) 1 dan format JSON *set_output*. Untuk memastikan instruksi benar-benar tereksekusi di tingkat fisik, sistem menerapkan **5 tahapan status transaksional**:

1. **pending:** Perintah berhasil dibuat dan tersimpan di basis data MariaDB.
2. **sent:** Perintah sukses dipublikasikan oleh *MQTT Writer* ke MQTT Broker.
3. **acked:** ESP32 membalas ke topik *smartfarm/{node_id}/confirm*. Komponen *subscriber* mencocokkan UUID *req_id* balasan dengan data transaksi lokal.
4. **timeout:** Jika dalam batas waktu 8 detik tidak ada balasan konfirmasi dari ESP32, komponen **Sweep Worker** (goroutine latar belakang) secara otomatis mengubah status transaksi menjadi *timeout*.
5. **failed:** Status langsung dikategorikan *failed* jika terjadi kegagalan jaringan saat mempublikasikan pesan (misalnya broker MQTT terputus).

3.8 Penambahan Layanan Pengolahan Citra

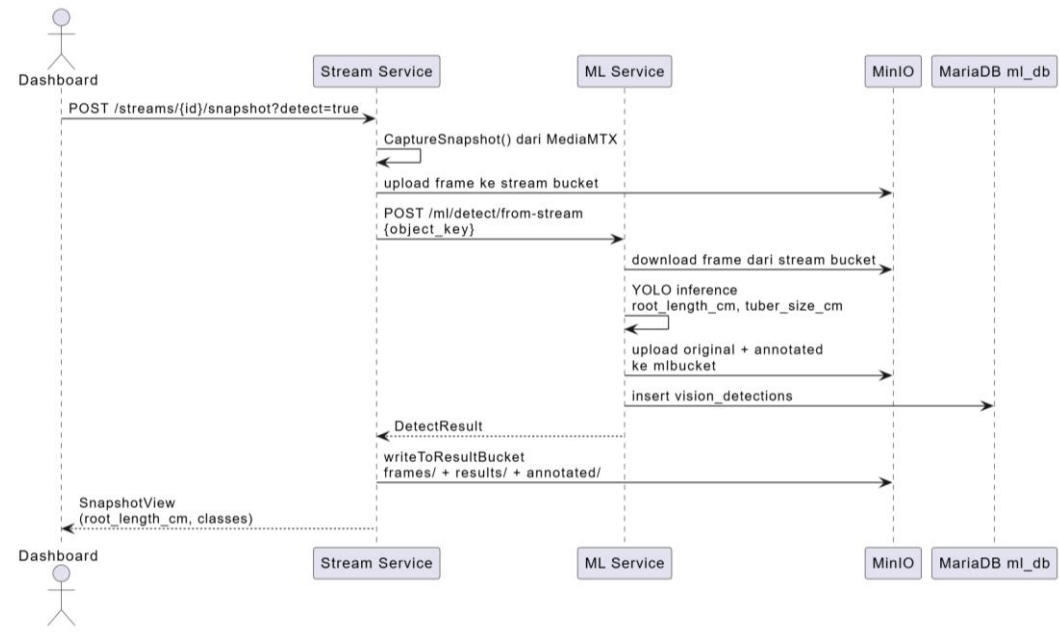
Untuk mekanisme deteksi akar dan umbi diperlukan model pengolahan citra. Model tersebut sudah tersedia dan berbasis YOLO (*You Only Look Once*) arsitektur *single-stage object detector* dari pustaka Ultralytics Model yang telah siap ini dikelola oleh ML Service, yang memiliki fungsi ganda sebagai penyimpan model dan penyedia API untuk mengeksekusi inferensi gambar. Disediakan juga *Environment Variable* Lampiran G.1 untuk melakukan konfigurasi awal kalibrasi pixel/cm, bucket, dan konfigurasi lainnya sebelum layanan dijalankan.



Gambar 3.13 Class Diagram — Model Registry

ModelRegistry mengelola siklus hidup model YOLOv8 dengan metadata yang tersimpan di MariaDB vision_models. Model di-load secara lazy dan di-cache dalam _loaded dict yang dilindungi threading.Lock. Saat model dipanggil, load() memeriksa cache; jika belum ada, bobot.pt diverifikasi di /app/models lalu diinisialisasi via YOLO(path). Pembaruan model menghapus cache lama (_loaded.pop(model_id)) agar model baru langsung aktif tanpa downtime. VisionDetection menyimpan hasil inferensi dengan URL gambar

original dan annotated di MinIO, serta metadata deteksi seperti root_length_cm, tuber_size_cm, confidence, dan waktu eksekusi.



Gambar 3.14 Sequence Diagram — AI Detect Inference Flow

AI Detect pada Live Dashboard memicu POST /streams/{id}/snapshot?detect=true. Stream Service menangkap frame dari MediaMTX, mengunggahnya ke MinIO bucket stream, lalu memanggil ML Service POST /ml/detect/from-stream dengan object_key. ML Service mengunduh frame, memuat model YOLOv8 via ModelRegistry (dengan cache in-memory), menjalankan inferensi untuk mendeteksi akar dan umbi, mengukur panjang akar dalam cm, lalu menyimpan gambar original dan annotated ke MinIO mlbucket. Hasil deteksi juga disimpan ke MariaDB vision_detections. Stream Service kemudian menulis salinan frame, JSON hasil, dan gambar annotated ke mlbucket sebelum mengembalikan SnapshotView ke Dashboard.

3.9 Algoritma Misting Adaptif dan Layanan Kontrol Adaptif

Pengembangan layanan kontrol adaptif diawali dengan melatih model Reinforcement Learning kemudian pengembangan layanan kontrol adaptif.

3.9.1 Algoritma Misting Adaptif

Model berupa Reinforcement Learning berbasis algoritma TD3 (Twin Delayed Deep Deterministic Policy Gradient) disimulasikan dengan menggunakan pustaka Python Gymnasium. Lingkungan simulasi dirancang semirip mungkin dengan kondisi fisik aeroponik nyata, lengkap dengan pengkondisian cuaca ekstrem serta noise sensor untuk menguji ketahanan model. Pelatihan ini bertujuan agar sistem mampu mengoptimalkan dan memperbarui durasi penyemprotan nutrisi secara otomatis berdasarkan dinamika lingkungan.

Model TD3 bertindak sebagai pengambil keputusan yang membaca kondisi sistem dengan data 10 dimensi Ruang Keadaan (State Space). Berdasarkan data tersebut, model secara dinamis menentukan 3 dimensi Ruang Aksi (Action Space), yaitu mengatur durasi pengkabutan (misting), menentukan interval jeda penyemprotan, serta mengendalikan status buka-tutup katup (valve).

Ruang Keadaan (State Space) — 10 Dimensi:

Dimensi	Variabel	Sumber Data (Deployment)
1	L_root — panjang akar (cm)	MinIO metadata deteksi ML
2	U_status — skor kondisi tanaman [0,1]	MinIO metadata deteksi ML
3	T_in — suhu dalam ruang	Telemetry Module Service
4	H_in — kelembapan dalam ruang	Telemetry Module Service
5	T_out — suhu luar ruang	Telemetry Module Service
6	H_out — kelembapan luar ruang	Telemetry Module Service
7	EC — electrical conductivity	Telemetry Module Service
8	pH — keasaman larutan nutrisi	Telemetry Module Service
9	T_nut — suhu larutan nutrisi	Telemetry Module Service
10	I_day — indeks intensitas matahari	Kalkulasi berdasarkan jam lokal

Ruang Aksi (Action Space) — 3 Dimensi:

Dimensi	Aksi Ternormalisasi [-1,1]	Rentang Fisik
a_mist	Durasi misting	D_mist dalam [120, 600] detik
a_interval	Jeda antar misting	interval_sec dalam [120, 600] detik
a_valve	Buka/tutup valve	A_valve: 0=OFF, 1=ON (threshold: nilai $\geq 0 \rightarrow$ ON)

Model TD3 dilatih menggunakan pendekatan **dense reward shaping** agar agent belajar mengambil keputusan kontrol yang optimal, seimbang, dan aman. Sinyal reward dirancang menjadi tiga kategori utama:

1. **Insentif Utama & Stabilitas Lingkungan:** Agent diberi reward berdasarkan pertumbuhan fisik tanaman yang dinormalisasi dengan kesesuaian lingkungan. Untuk mengatasi sparse reward, sinyal padat (*dense signal*) diberikan setiap langkah berbasis faktor pembatas lingkungan (pH, EC, kelembapan, suhu udara/akar, dan oksigen).
2. **Bonus Kelangsungan Hidup (Survival Bonus):** Agent mendapatkan bonus bertingkat seiring berjalannya waktu (+0,5/langkah hingga +200 pada milestone 18 jam). Jika episode mati prematur akibat kondisi kritis, agent dikenakan penalti keras.
3. **Efisiensi Sumber Daya & Keragaman Aksi:** Memberikan cost pada penggunaan valve serta insentif untuk durasi misting lebih singkat dan interval lebih panjang. Penalti diberikan jika aksi agent terlalu monoton (*mode collapse*) atau berisiko tinggi.

Fungsi reward total dirancang untuk memprioritaskan pertumbuhan tanaman, stabilitas lingkungan, efisiensi sumber daya, dan eksplorasi aksi yang beragam. Total reward dihitung sebagai:

$$\begin{aligned}
 R_{\{total\}} = & R_{\{growth\}} + R_{\{growth_proxy\}} + R_{\{state\}} + P_{\{diversity\}} + R_{\{efficiency\}} \\
 & - C_{\{resource\}} - P_{\{env\}} - P_{\{hypoxia\}} - P_{\{extreme\}} - P_{\{shrink\}} \\
 & - P_{\{death\}}
 \end{aligned}$$

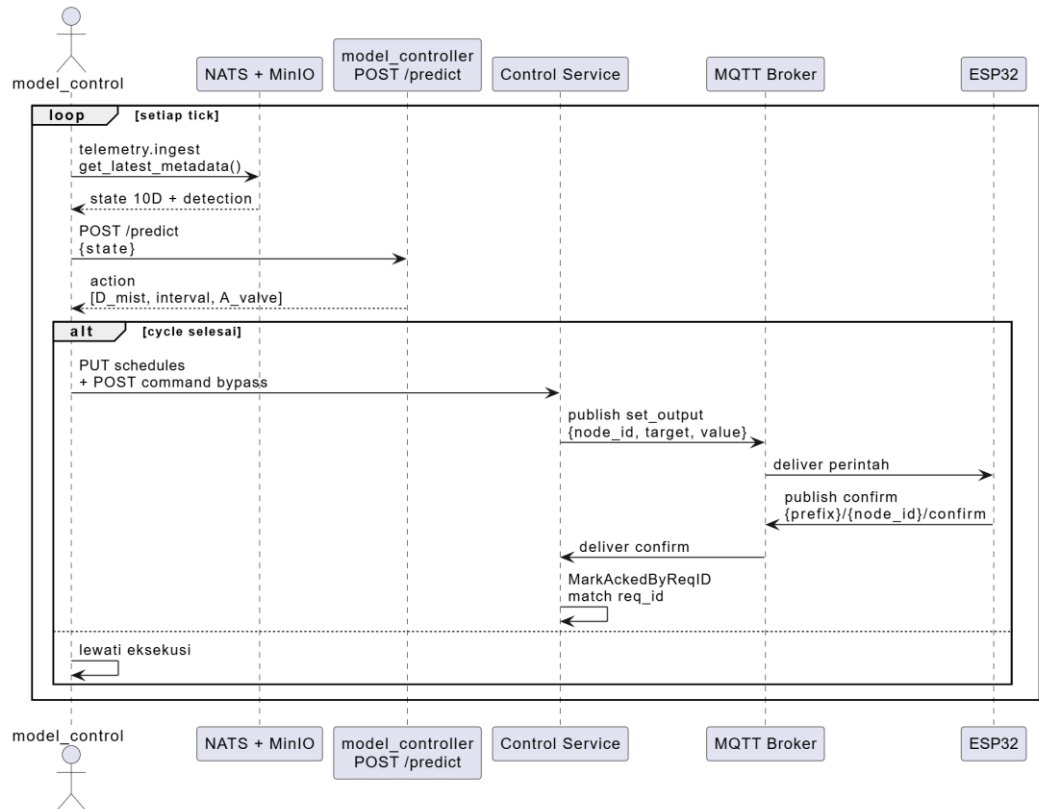
Untuk detail lengkap Reward setiap parameter terdapat pada Lampiran ...

Kemudian model dilatih sebanyak 2 juta *timesteps* menggunakan kapasitas *replay buffer* sebesar 2 juta data. Kompleksitas simulasi dinamika lingkungan aeroponik serta besarnya volume sampel pengalaman yang diproses dalam *buffer* membuat iterasi pelatihan ini memakan waktu lebih dari satu hari penuh. Yang menghasilkan dua artefak utama, yaitu bobot model TD3 yang telah teroptimasi (*aerponic_td3.zip*) dan vektor normalisasi data (*vec_normalize*). Kemudian model tersebut dimuat oleh *model-controller* untuk mengeksekusi inferensi kontrol msiting secara presisi.

3.9.2 Layanan Kontrol Adaptif

Pada layanan kontrol adaptif mengintegrasikan dua layanan kustom baru untuk menguji algoritma kontrol alternatif: **model-control** dan **model-controller**. Kedua layanan ini bertindak sebagai entitas mandiri yang sepenuhnya terpisah dari sistem inti:

1. **model-control**: Layanan kustom berbasis Python yang mengelola pelatihan model pembelajaran penguatan (Reinforcement Learning) serta memproses keputusan logika kontrol berbasis AI secara otonom.
2. **model-controller**: Layanan perantara (*controller*) kustom yang bertindak sebagai jembatan untuk mengekspos API prediksi/kontrol model ke dunia luar dan berinteraksi dengan API Gateway Kong.



Gambar 3.15 Sequence Diagram — Adaptive Control Flow

Berikut alur singkat kontrol adaptif dari MinIO/telemetry hingga ESP32:

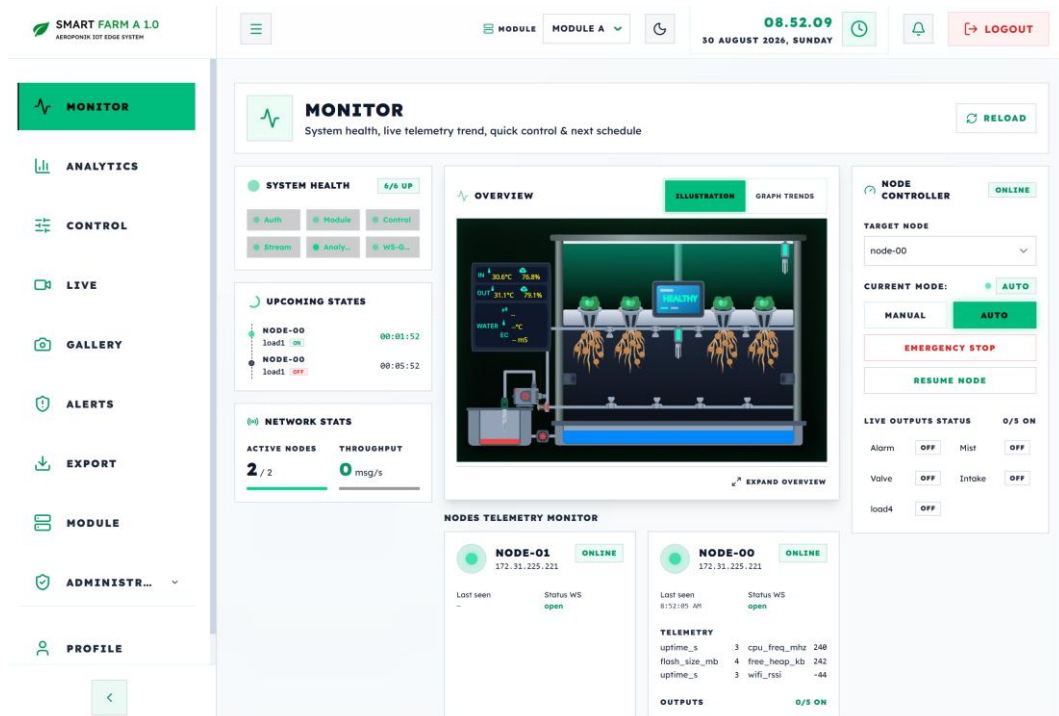
1. Data dikumpulkan baik telemetry ataupun metadata dari gambar model-control subscribe ke subject NATS telemetry.ingest untuk menerima data sensor real-time dari Module Service (suhu, kelembapan, EC, pH, dll.) model-control membaca metadata deteksi YOLOv8 (root_length_cm, condition) yang disimpan oleh ML Service di MinIO mlbucket/detected/.
2. Kemudian keseluruhan data disusun menjadi vektor state 10 dimensi: [L_root, U_status, T_in, H_in, T_out, H_out, EC, pH, T_nut, I_day]
3. Selanjutnya model-control memanggil model-controller untuk Inferensi TD3 via POST /predict secara periodik (setiap 5 detik) dengan state 10D. model-controller mengembalikan aksi D_mist: durasi misting (120–600 detik), interval_sec: jeda antar misting (120–600 detik) dan A_valve: kontrol valve ON/OFF

4. Aksi baru disimpan sebagai pending_action. model-control mengecek apakah siklus pompa aktif saat ini sudah selesai. Jika ya, aksi diterapkan; jika tidak, siklus berjalan sampai selesai untuk mencegah mid-cycle reset.
5. Ketika pending sudah selesai model-control memperbarui jadwal via Control Service API (POST /v1/control/schedules). Juga model-control mengirim perintah langsung via Control Service API untuk kontrol valve.
6. Kemudian Control Service mempublikasikan perintah MQTT ke topik {prefix}/actuator/{node_id} dengan payload:

```
{
  "action" : "set_output",
  "target" : "pump",
  "value" : 1,
  "req_id" : "<uuid>"
}
```

7. Terakhir perintah diterima dan dieksekusi oleh ESP32 kemudian mengirim feedback konfirmasi ke {prefix}/{node_id}/confirm.

3.10 Dashboard



Gambar 3.16 Dashboard

Agar pengguna dapat berinteraksi dengan server dibuat antarmuka Dashboard agar mempermudah pengguna dalam monitoring maupun konfigurasi sistem. Semua data mengalir dari dashboard ke backend melalui Kong API dan WebSocket, dashboard tidak pernah menyentuh layanan secara langsung.

Berikut fitur yang terdapat pada Dashboard:

1. Real-time tanpa polling telemetry dan status di-push via WebSocket (WS-Gateway), sehingga grafik ikut bergerak saat data baru tiba.
2. Fitur utama monitoring (grafik telemetry hidup), analitik (riwayat & ekspor CSV), kontrol (perintah manual, jadwal otomatis, tombol emergency stop), live stream, galeri (snapshot/kamera), manajemen perangkat, dan manajemen profil
3. Peran (RBAC) admin/operator boleh mengontrol; viewer hanya boleh memantau analitik, galeri, dan profil (tidak bisa mengirim perintah).

Dengan pemisahan ini, nantinya developer dapat memperbarui atau mengganti tampilan dashboard tanpa mempengaruhi sistem yang sudah berjalan di backend.

3.11 Skenario Pengujian

Pengujian performa sistem dilakukan secara bertahap dan progresif melalui skenario berikut:

1. Pengujian Modularitas Perangkat Tepi: Validasi asimilasi sensor/aktuator baru secara dinamis melalui Captive Web Portal tanpa proses rekompilasi.
2. Pengujian Integrasi End-to-End: Validasi kelancaran pipa data dari ESP32, Module Service, NATS JetStream, hingga visualisasi dasbor dan eksekusi kontrol.
3. Evaluasi Tahap 1 (Konfigurasi Minimal): Pengukuran performa dasar (baseline) jaringan dan komputasi tanpa beban algoritma kecerdasan buatan (AI).

4. Evaluasi Tahap 2 (Konfigurasi Penuh): Pengujian sistem di bawah beban komputasi AI penuh, mencakup inferensi YOLOv8 dan kontrol adaptif RL TD3 secara simultan.
5. Analisis Komparatif: Komparasi hasil kedua tahap pengujian untuk menganalisis dampak integrasi AI terhadap efisiensi, stabilitas iklim mikro, dan overhead jaringan.
6. Kesimpulan Awal: Sintesis hasil untuk memverifikasi efektivitas paradigma modularitas ganda terhadap kemudahan pengembangan dan keandalan otomatisasi sistem.

BAB IV

HASIL PENELITIAN

Bab ini menyajikan hasil implementasi, pengujian bertahap dan pembahasan komprehensif sistem otomatisasi aeroponik terdistribusi yang berfokus pada **modularitas dan skalabilitas**.

4.1 Pengujian Modularitas Sensor, Aktuator & Konfigurasi Dinamis via Antarmuka Web Captive Portal

Pertama perangkat ESP32 harus di-*flash* menggunakan versi *firmware* terbaru yang tersedia pada repositori GitHub untuk petunjuk *flashing* ada di Lampiran J.1. Proses pengunggahan ini dapat dilakukan menggunakan *Flash Tool* atau melalui fitur *Web Flash* pada [Espressif ESPTool Online](#).

4.1.1 Skenario Penambahan Sensor dan Aktuator

Pada tahap pengujian awal, *node* sensor ESP32 berhasil dikonfigurasi untuk mendukung berbagai jenis sensor, seperti **digital**, **analog**, **Modbus**, dan **I2C**. Selain itu, perangkat ini juga mampu mengendalikan aktuator menggunakan sinyal **digital** maupun **PWM**.

Pengujian dilakukan di dua lokasi berbeda dengan penyesuaian sebagai berikut:

- **Screenhouse:** Pengujian difokuskan pada lingkungan aeroponik menggunakan konfigurasi sensor dan aktuator yang relevan.
- **Lab PTIO ITB:** Pengujian dilakukan khusus untuk sensor I2C (INA219) guna mengukur parameter daya yang saat ini belum diterapkan di *screenhouse*.

Meski dilakukan di lokasi yang berbeda, kedua pengujian ini tetap konsisten karena menggunakan basis *firmware* yang sama. Perbedaan hanya terletak pada keberadaan modul sensor I2C.

Sebelum dilakukan pengujian perangkat harus dikonfigurasi terlebih dahulu agar terhubung ke broker untuk petunjuk terdapat pada Lampiran J.2. Selanjutnya pengguna dapat langsung melakukan konfigurasi sensor maupun aktuator.

Pada pengujian di lokasi pertama terdapat beberapa sensor digital dan modbus, untuk aktuator hanya digital berikut adalah hasil konfigurasi melalui antarmuka:

The screenshot displays a web-based interface for configuring GPIO inputs. It features a list of four existing inputs, each with a name, GPIO pin, input type, pull resistor, interrupt mode, and debounce time. Below this list is a form to create a new input, including fields for GPIO Pin, Input Type, Pull Resistor, Interrupt Mode, Debounce (ms), Name Label, and an Invert Logic checkbox. A 'Done' button is present at the bottom right of the form, and a '+ Add Input' button is at the bottom left.

GPIO Pin	Input Type	Pull Resistor	Interrupt Mode	Debounce (ms)
0	DIGITAL	NONE	NONE	0
1	DIGITAL	NONE	NONE	0
2	DIGITAL	NONE	NONE	0
3	DIGITAL	NONE	NONE	0

Form fields for 'New Input':

- GPIO Pin: 0
- Input Type: DIGITAL
- Pull Resistor: NONE
- Interrupt Mode: NONE
- Debounce (ms): 0
- Name Label: New Input
- Invert Logic: ☐ LOW = Active

Gambar 4.1 Konfigurasi Input GPIO

Output

load1
GPIO 25 | DIGITAL

EditRemove

load2
GPIO 26 | DIGITAL

EditRemove

load3
GPIO 27 | DIGITAL

EditRemove

load4
GPIO 14 | DIGITAL

EditRemove

buzzer
GPIO 33 | DIGITAL

EditRemove

GPIO Pin	Output Type	Name Label	
0	DIGITAL	New Output	Done

+ Add Output

Gambar 4.2 Konfigurasi Output GPIO

Sensor Configurations

cwt1
ID: 1 | Baud: 9600 | Regs: 2

EditRemove

cwt2
ID: 2 | Baud: 9600 | Regs: 2

EditRemove

Sensor Name	Slave ID	Baudrate
New RS485 Sensor	1	9600

Registers to Read

0	HOLDING	new_reg	1	X
---	---------	---------	---	---

+ Reg

Done

+ Add Modbus Sensor

Save & Reboot

Gambar 4.3 Konfigurasi Sensor Modbus

Untuk pengujian pada sistem Aeroponik di Screenhouse didapatkan data Telemetry seperti berikut:

```
{
  "node_id": "ECE334219870",
  "fw_version": "1.0.0",
  "network": {
    "ssid": "Aeroponik 1",
    "ip_address": "192.168.1.100",

```

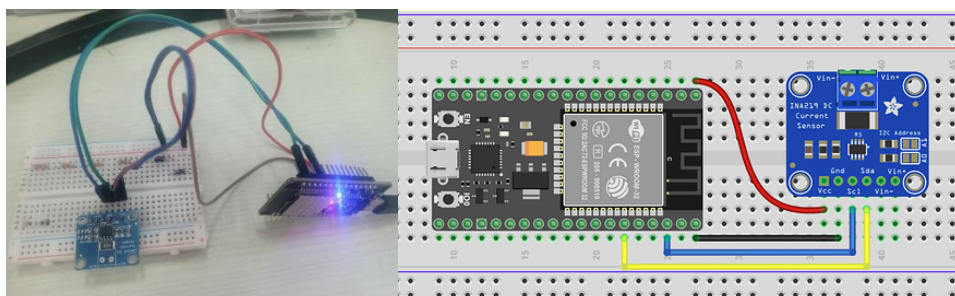
```

    "wifi_rssi": -44
  },
  "device_info": {
    "uptime_s": 1549733,
    "cpu_freq_mhz": 240,
    "free_heap_kb": 166,
    "flash_size_mb": 4
  },
  "connection_stats": {
    "mqtt_connected": true,
    "uptime_s": 1549733
  },
  "telemetry": {
    "inputs": {
      "input1": 0,
      "input2": 1,
      "input3": 1,
      "input4": 1
    },
    "outputs": {
      "load1": 0,
      "load2": 0,
      "load3": 0,
      "load4": 0,
      "buzzer": 0
    },
    "modbus": {
      "cwt1": {
        "hum": 45.70000076,
        "temp": 31.89999962
      },
      "cwt2": {
        "hum": 44.29999924,
        "temp": 32.40000153
      }
    }
  }
}

```

Hasil ini menunjukkan bahwa semua data sensor dan aktuator dapat terbaca dan berhasil publish melalui MQTT.

Semua pengujian diatas dilakukan di lingkungan aeroponik, selanjutnya untuk pengujian penambahan perangkat I2C dilakukan di Lab PTIO ITB



Gambar 4.4 Wiring Diagram Sensor INA219

Pengujian ini menggunakan satu unit ESP32 dengan firmware yang sama kemudian dihubungkan langsung ke modul **INA219** melalui jalur komunikasi I2C (pin SDA dan SCL). Konfigurasi dilakukan langsung melalui antarmuka web dengan memasukkan parameter nama sensor, tipe sensor, alamat I2C, serta alokasi pin SDA dan SCL.

Gambar 4.5 Konfigurasi Sensor I2C

Ketika tombol "Save & Reboot" ditekan pada antarmuka, server web internal ESP32 mengompilasi parameter formulir dan menyimpannya di config.json.

Payload JSON telemetry yang dipancarkan ESP32 ke topik MQTT *smartfarm/8C94DF6BCDFC/telemetry* secara otomatis bertambah dan memuat data dari sensor baru:

```
{
  "node_id": "8C94DF6BCDFC",
  "fw_version": "1.0.0",
  "network": {
    "ssid": "PAU Hotspot",
    "ip_address": "10.18.128.242",
    "wifi_rssi": -73
  },
  "device_info": {
    "uptime_s": 1493,
    "cpu_freq_mhz": 240,
    "free_heap_kb": 159,
    "flash_size_mb": 4
  },
  "connection_stats": {
    "mqtt_connected": true,

```

```
    "uptime_s": 1493
  },
  "telemetry": {
    "outputs": {},
    "i2c": {
      "power_monitor": {
        "bus_voltage_v": 1.011999965,
        "shunt_voltage_mv": 0.029999999,
        "current_ma": 0.100000001,
        "power_mw": 0
      }
    }
  }
}
```

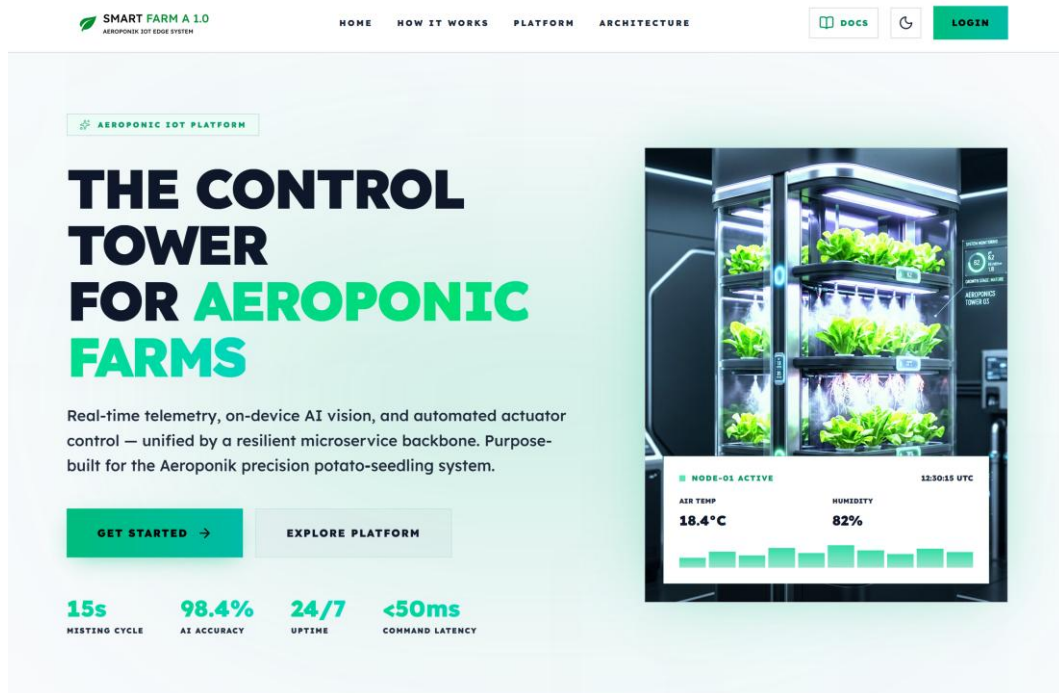
Secara keseluruhan pengguna cukup berinteraksi dengan antarmuka *Captive Web Portal* tanpa menyentuh baris kode C++ maupun berkas JSON mentah.

4.2 Pengujian Alur Integrasi End-to-End via Antarmuka Dashboard

Setelah lapisan firmware terverifikasi mampu mengirimkan data secara dinamis, pengujian dilanjutkan ke alur operasional end-to-end yang dilakukan langsung melalui antarmuka *Dashboard*.

4.2.1 Registrasi Module, Node, dan Tag Mapping via Dashboard

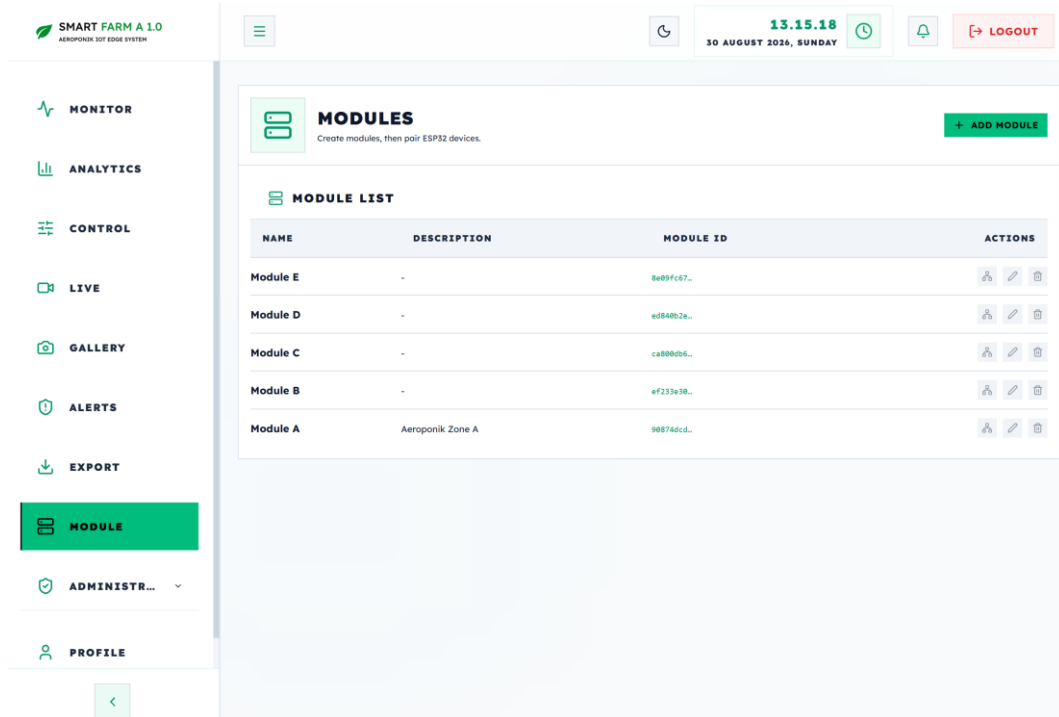
Untuk persiapan sebelum mendaftarkan perangkat, service wajib dijalankan dengan *docker compose up -d* untuk menjalankan semua service sekaligus. Kemudian *docker ps* untuk melihat apakah semua service sudah berjalan normal, jika sudah dashboard dapat diakses melalui *browser* dengan *url* localhost:5173.



Gambar 4.6 Tampilan Landing Page Dashboard

Gambar 4.7 Tampilan Halaman Login

Pengguna mengarahkan browser ke halaman login dan memasukkan kredensial administrator default *username* = admin dan *password* = admin1234.



Gambar 4.8 Tampilan Halaman Module

Pada menu *Module*, pengguna menekan tombol *Add Module*, memasukkan identitas *Modul A* dan deskripsi (opsional), dan *submit*.



Gambar 4.9 Auto Discovery

PAIRED (2)

 **NODE-01**

Node ID: node-01

IP: 172.31.225.221

FW: 1.0.0

Last Seen: 39s ago

CONFIGURE

UNPAIR



 **NODE-00**

Node ID: node-00

IP: 172.31.225.221

FW: 1.0.0

Last Seen: 10s ago

CONFIGURE

UNPAIR



Gambar 4.10 Tampilan Status Perangkat yang Sudah Terhubung

Ketika node ESP32 (ECE334219870) dinyalakan, pesan status online dikirim ke MQTT Broker. Module Service mendeteksi node baru berstatus *Unpaired*. Pengguna menekan tombol *PAIR* yang ada pada menu *Node Configuration*, yang secara otomatis mengeksekusi POST `/v1/nodes/node-01/pair`, dan mengubah status perangkat menjadi *Paired*.

 **TELEMETRY MAPPING**

Firmware publishes outputs as `telemetry.<name>.<name>` (e.g. `telemetry.inputs.input1`). Map them to a DB tag — the stored key is the raw output name (`telemetry.inputs.input1`) sent back to the firmware. Mapped telemetry appear as targets in the **Analytics** menu. Use **Detect Keys** to auto-scan from live MQTT.

+ ATTACH TELEMETRY

MQTT key

DB tag

Unit

float

+ ADD ROW

DETECT KEYS

MQTT KEY	DB TAG	DISPLAY NAME	UNIT	TYPE	ON
connection_stats.mqtt_connecte	connection_stats.mqtt_connected	connection_stats.mqtt_connected		bool	☐
connection_stats.uptime_s	connection_stats.uptime_s	connection_stats.uptime_s		int	☐
device_info.cpu_freq_mhz	device_info.cpu_freq_mhz	device_info.cpu_freq_mhz		int	☐
device_info.flash_size_mb	device_info.flash_size_mb	device_info.flash_size_mb		int	☐
device_info.free_heap_kb	device_info.free_heap_kb	device_info.free_heap_kb		int	☐
device_info.uptime_s	device_info.uptime_s	device_info.uptime_s		int	☐

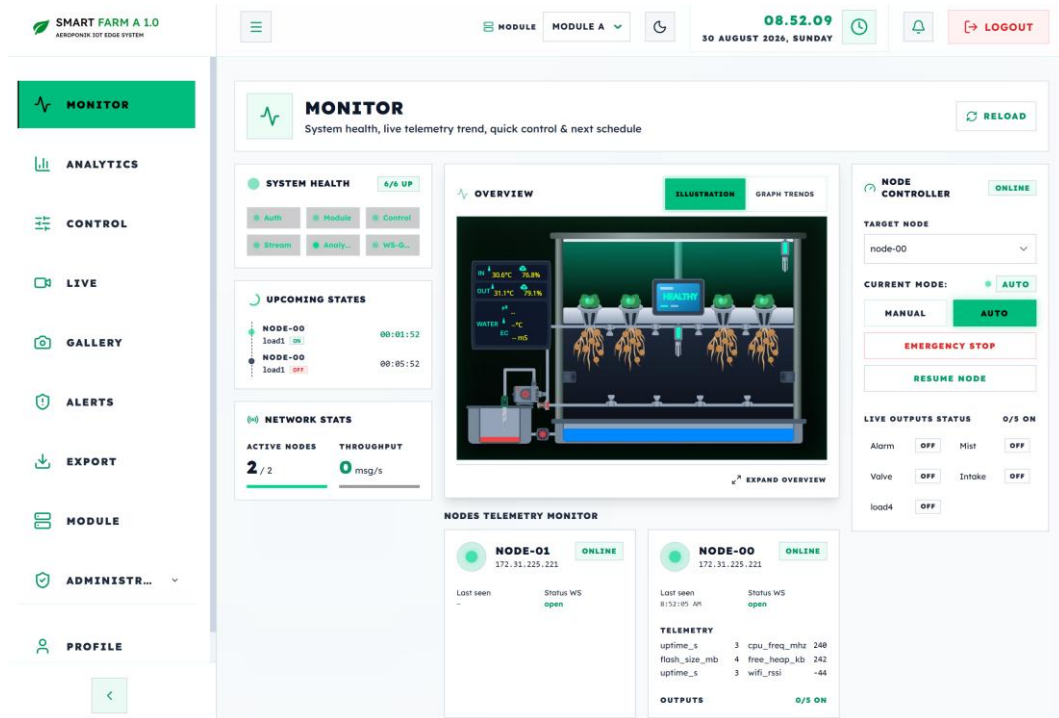
Gambar 4.11 Tampilan Halaman Tag Mapping

Agar data telemetri dapat disimpan di database pengguna perlu memetakan jalur telemetri ke database. Pengguna dapat membuka *Tag Mapping Modal*, kemudian pengguna bisa langsung menekan tombol *Detect Keys* dan *Detect Ouput* untuk otomatis mendeteksi metrik baik sensor maupun aktuator. Secara default mapping akan disesuaikan dengan metrik telemetrinya, namun pengguna dapat menyesuaikannya dan mengubah kolom display name dan kolom unit untuk satuan.

48

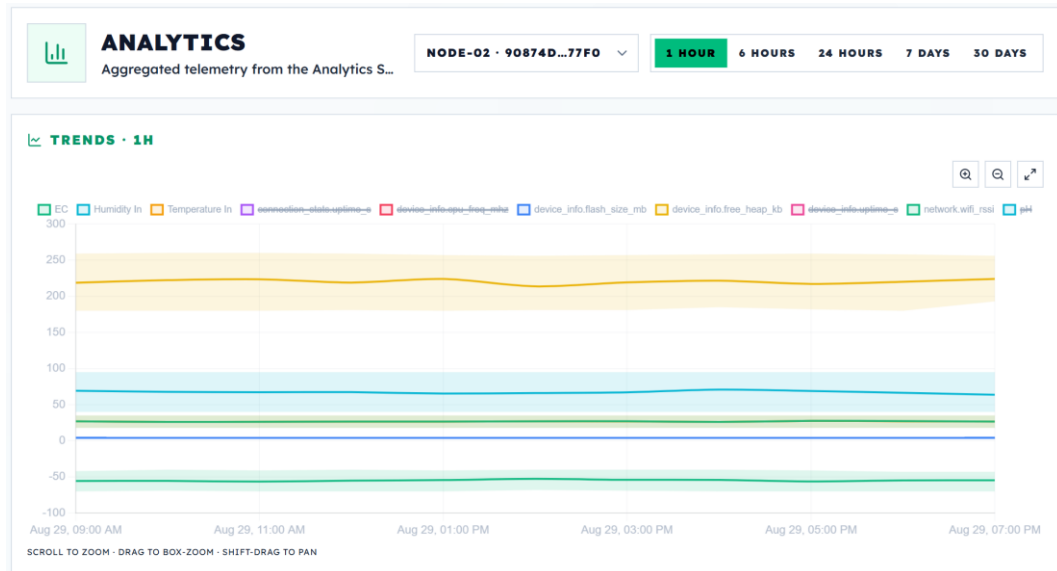
Penamaan ini nantinya akan digunakan untuk kemudahan pembacaan grafik di page *Analytics* maupun penamaan aktuator pada page *Control*.

4.2.2 Verifikasi Tampilan Dashboard, Analytics dan Pengujian Control



Gambar 4.12 Tampilan Halaman Monitor

Pada halaman *Monitor*, data telemetri yang sudah di *mapping* langsung ditampilkan pada section overview begitu juga dengan kondisi aktuator. Data diterima melalui koneksi WebSocket persisten WS-Gateway (/ws) dengan latensi yang rendah.

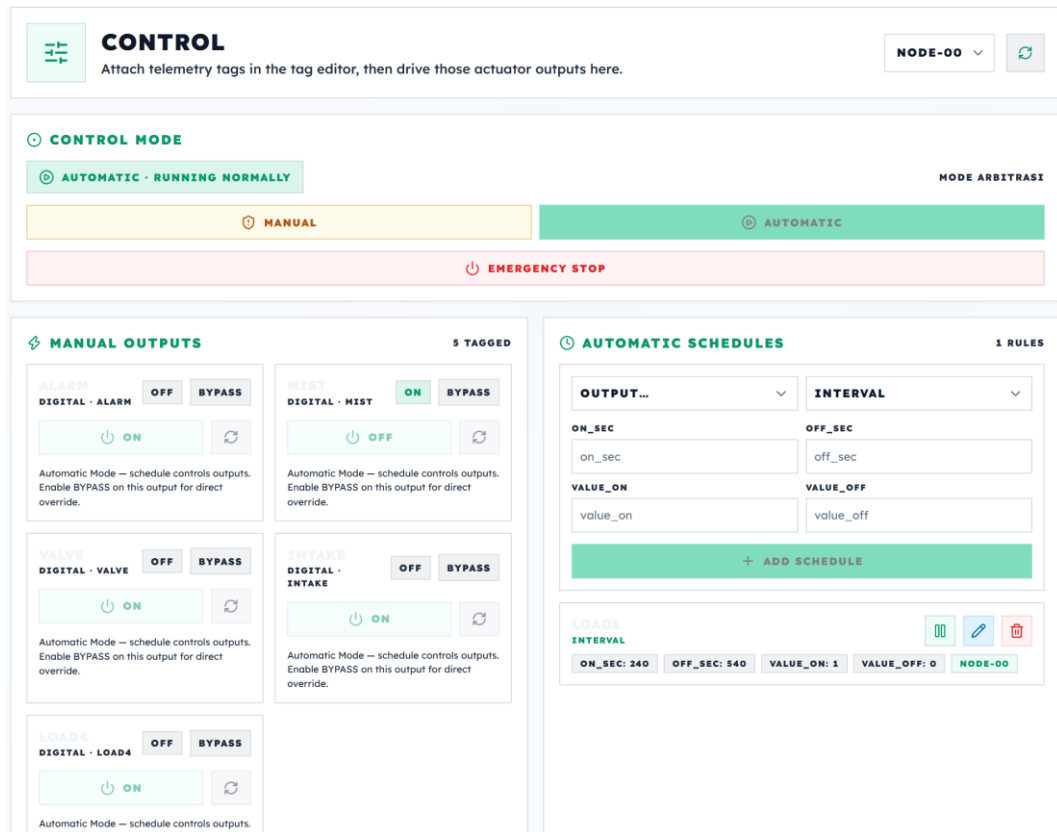


Gambar 4.13 Grafik Historis Telemetri



Gambar 4.14 Hasil Pengolahan Data Halaman Analytics

Data telemetri mentah tersimpan dalam *hypertable* TimescaleDB dan diolah secara agregat pada halaman *Analytics*, menghasilkan grafik historis fluktuasi iklim mikro.



Gambar 4.15 Tampilan Halaman Control

Melalui halaman *Control*, operator dapat menguji kontrol aktuator, secara default sistem akan menjalankan mode otomatis yang mengikuti penjadwalan. Kemudian mode dipindah manual untuk bisa mengaktifkan langsung.

Ketika tombol diaktifkan dashboard akan mengirimkan request `POST /v1/control/commands` via Kong Gateway. Kemudian Control Service meneruskan instruksi ke ESP32 melalui topik MQTT `smartfarm/actuator/ ECE334219870`. ESP32 akan mengaktifkan relai aktuator fisik dan mengirimkan sinyal konfirmasi ke `smartfarm/ECE334219870/confirm`.

COMMAND LOG				100 ENTRIES
OUTPUT	TYPE	VALUE	STATUS	TIME
LOAD1	INTERVAL	1	acked	1:40:47 PM
LOAD1	INTERVAL	0	acked	1:31:47 PM
LOAD1	INTERVAL	1	acked	1:27:45 PM
LOAD1	INTERVAL	0	acked	1:18:45 PM
LOAD1	INTERVAL	1	acked	1:14:44 PM
LOAD1	INTERVAL	0	acked	1:05:44 PM
LOAD1	INTERVAL	1	acked	1:01:43 PM
LOAD1	INTERVAL	0	acked	12:52:42 PM
LOAD1	INTERVAL	1	acked	12:48:40 PM
LOAD1	INTERVAL	0	acked	12:39:40 PM

PREV
1 / 10
NEXT

Gambar 4.16 Tampilan Command Log Konfirmasi Aktuator

Control Service memvalidasi konfirmasi dan mengembalikan status sukses ke dasbor. Indikator sakelar pompa pada dasbor berubah warna menjadi hijau tertanda aktif.

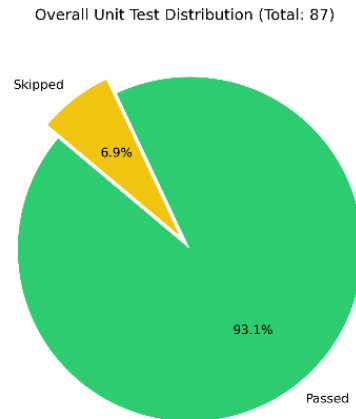
4.3 Evaluasi Tahap I: Konfigurasi Minimal tanpa Service Tambahan

Pengujian performa dan ketahanan sistem dilakukan secara bertahap. Tahap I menetapkan performa dasar sistem ketika hanya menjalankan layanan inti (Auth, Module, Analytics, Control, Stream, Alert, Notification, WS-Gateway, Kong, DBs, NATS, MQTT) dengan beban kerja 1 Modul dan 4 Node ESP32 yang diaktifkan.

4.3.1 Pengujian Unit Test Fitur Layanan

Sebagai persiapan awal dilakukan pengujian unit test untuk memastikan semua fitur berjalan normal. Pengujian ini dilakukan dengan menjalankan program pengujian secara otomatis

```
python3 test/unit_test.py --phase2
```

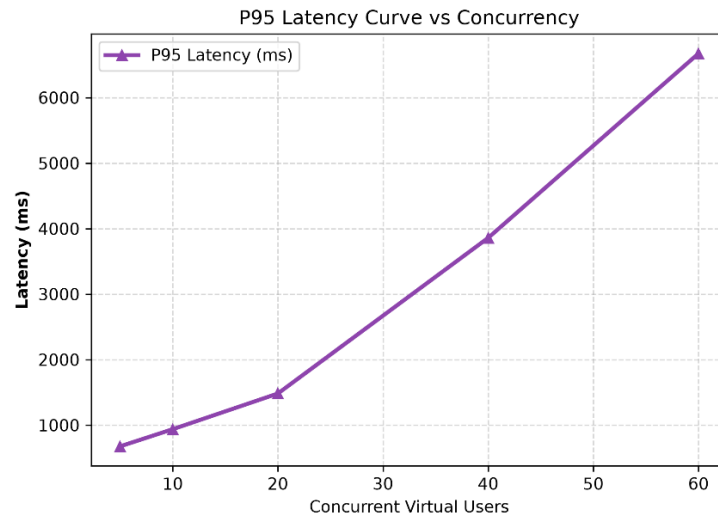
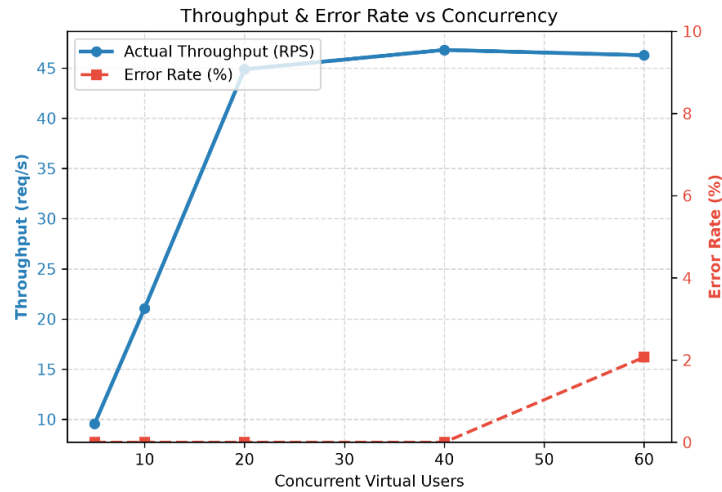


Secara keseluruhan dari 87 fitur yang ada sebanyak 93.1% Passed dan 6.9% Skipped. Ini membuktikan bahwa keseluruhan fitur yang diuji berjalan normal. Nilai 6.9% Skipped bukanlah indikator gagal melainkan karena beberapa fitur yang memerlukan data uji belum bisa didapatkan kemudian unit test tersebut diabaikan agar tidak mengganggu pengetesan selanjutnya.

4.3.2 Pengujian Stress Test

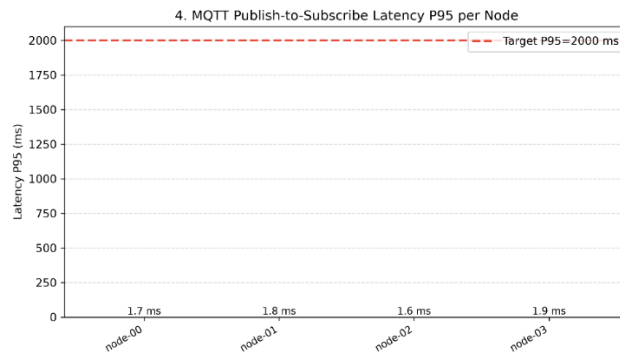
Pengujian stress test Tahap I dibagi menjadi dua bagian untuk membuktikan Skalabilitas Selektif dari dua sisi: (1) beban API pada gateway REST (Kong), dan (2) beban ingesti telemetry ujung-ke-ujung (MQTT → NATS → TimescaleDB).

Untuk uji beban API pengujian dilakukan pada layanan Kong API Gateway (/v1) terhadap 10 Layanan Inti. Pengujian ini dilakukan untuk mendapatkan **Breakpoint Capacity Test (Pencarian Batas Throughput)**: 5 level konkurensi (5, 10, 20, 40, 60 *users*) dengan target 10–500 RPS, masing-masing 8 detik.



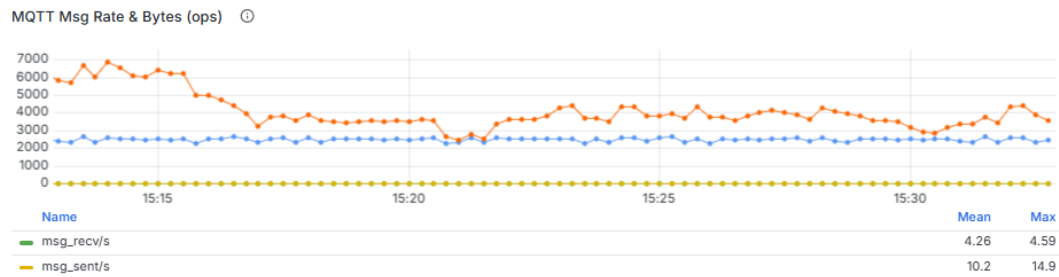
Hasil pengujian menunjukkan grafik *throughput* terhadap konkurensi naik cepat hingga mencapai puncak **51 RPS** pada Level 3 (20 *users*) dengan Latensi P95 **1.482 ms**. Sedangkan error baru muncul setelah 60 user sebesar di **2,1%**.

Uji beban di sisi ingesti telemetry diuji dengan beban 4 node dalam 1 module. Artinya dalam satu module terdapat 4 node yang masing-masing node membaca beberapa sensor. Pengujian ini untuk menguji Latensi P95 untuk publish-subscribe, publish throughput (msg/s Mosquitto); ingest throughput (in_msg/s NATS), *transaction* TimescaleDB.



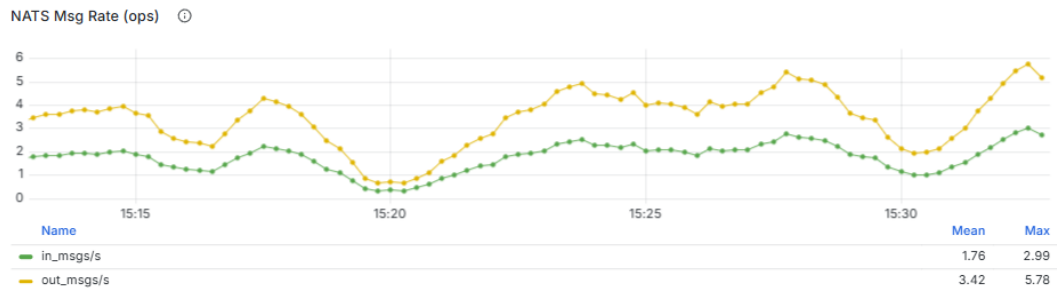
Gambar 4.17 Latensi Publish-Subscribe P95

Berdasarkan hasil pengujian latensi transmisi *publish-subscribe* yang dilakukan secara simultan menggunakan **empat node**, diperoleh rata-rata latensi P95 yang sangat rendah, yaitu **1.75 ms** dengan batas toleransi maksimum yaitu **2000 ms** (2 detik).



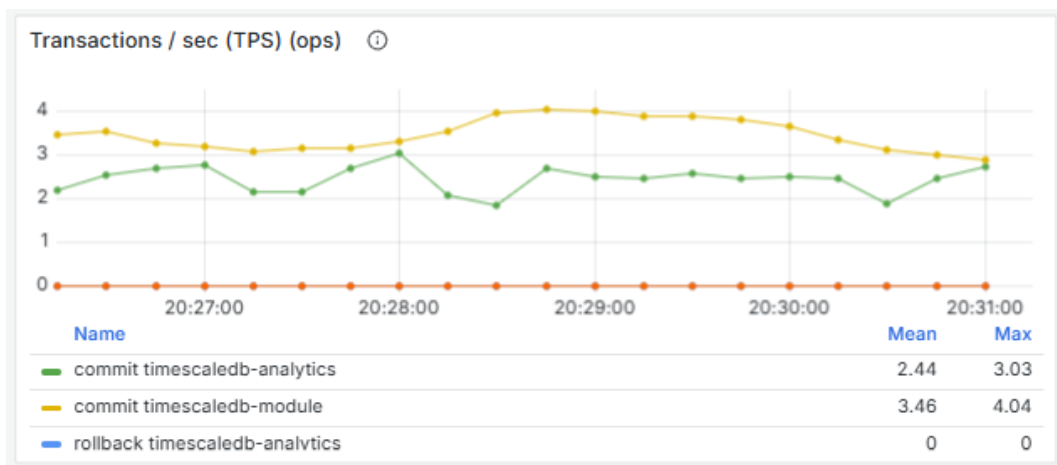
Gambar 4.18 Publish Troughput MQTT

Hasil pengujian troughput MQTT pada broker menunjukkan bahwa rata – rata troughput yang diterima broker adalah **4.26 msg/s**. Ini sesuai dengan konfigurasi node yang mengirimkan telemetri setiap 1 detik sekali.



Gambar 4.19 Ingest Throughput (in_msg/s NATS)

Berdasarkan grafik pengujian tersebut, rata-rata laju ingesti data stabil berada pada angka **1,76 msg/s** dengan beban puncak maksimum mencapai **2,99 msg/s**.



Gambar 4.20 Transaction in TimescaleDB (tps)

Gambar ini mengilustrasikan rata-rata laju transaksi komit pada basis data TimescaleDB oleh *Module Service* dalam jendela waktu 5 menit terakhir, dengan nilai rata-rata sebesar **3,46 tps** dan batas puncak maksimum **4,04 tps**. Untuk Analytics Service nilai rata-rata **2.44 tps** dan batas puncak maksimum **3.03 tps**.



Gambar 4.21 Utilisasi sistem saat stress test

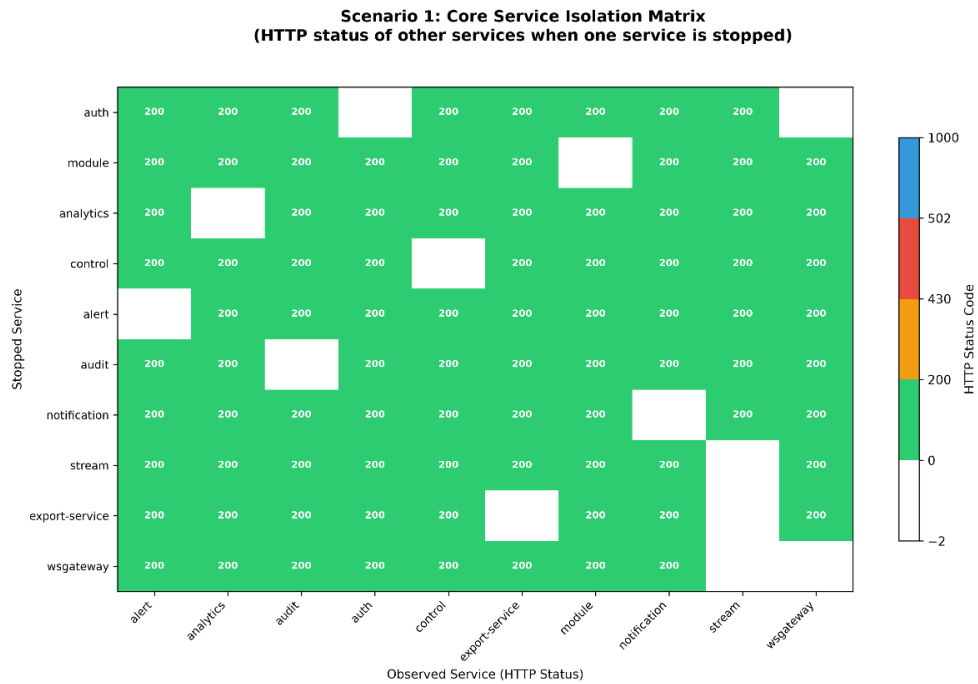
Selama fase pengujian beban, utilisasi sumber daya pada server *backend* menunjukkan efisiensi yang sangat tinggi dengan beban CPU puncak hanya

mencapai **9.69%** dan total konsumsi RAM keseluruhan sebesar **0.8 GB**. Di tingkat kontainer, rata-rata konsumsi memori per layanan mikro (*microservice*) berada pada angka yang sangat ringan, yaitu **28.1 MB**, dengan batas konsumsi puncak maksimum per layanan paling intensif terbatas sebesar **200 MB**.

4.3.3 Pengujian Resilience Chaos Test

Pengujian ini dilakukan untuk menganalisis Isolasi Kegagalan pada 10 Core Microservices yang berjalan tanpa AI/ML. Lima skenario chaos dirancang untuk mencoba memutuskan rantai komunikasi antar *bounded context* dan mengamati apakah kegagalan satu layanan merembet ke layanan lain (*cascade failure*):

1. Core Service Isolation Matrix: mematikan setiap layanan inti satu per satu dan memverifikasi bahwa layanan lain tetap beroperasi (HTTP 200). Menguji prinsip *failure domain isolation*.
2. Database & Cache Degradation Isolation: mematikan database individual (mariadb-module, mariadb-alert, mariadb-audit, mariadb-stream, mariadb-notification, redis-shared) untuk memastikan kegagalan database tidak menyeret service lain.
3. Kong Partial Backend Failure: mematikan beberapa layanan auxiliary sekaligus (alert, stream, audit, export-service) dan memverifikasi Kong tetap melayani healthy services.
4. Event Bus Blackhole & Reconnection: mematikan NATS broker selama 30 detik, memverifikasi REST API tetap berfungsi, dan memastikan auto-reconnection setelah recovery.
5. Cascading Failure Prevention: mematikan module service (paling terhubung) dan memverifikasi Auth, Analytics, Control, dan Alert tidak mengalami *cascade failure* (tidak return 500).



Gambar 4.22 Skenario pengujian 1 pada resilience cheos test

Secara keseluruhan, pengujian isolasi kegagalan pada 10 *core microservices* menunjukkan ketahanan sistem yang sangat baik. Seluruh skenario *chaos engineering* berhasil dilalui dengan kelulusan 100%, membuktikan bahwa arsitektur sistem mampu mencegah efek kegagalan berantai (*cascade failure*) dan menjaga ketersediaan layanan meskipun terjadi gangguan pada komponen lain. Untuk detail hasil pengujian pada skenario lain dapat dilihat di lampiran.

4.4 Evaluasi Tahap II: Konfigurasi Lengkap dengan Service Tambahan

Pada **Tahap II**, sistem dievaluasi pada kapasitas operasional penuh. Pengujian ini mengintegrasikan seluruh komponen tambahan Seperti ML, model-control, dan model- controller. Untuk detail konfigurasi dapat dilihat di lampiran.

4.4.1 Integrasi ML, model-control, model-controller

Pengujian ini membuktikan bahwa komponen kontrol adaptif bekerja secara *end-to-end*: dari deteksi visual, inferensi, penggabungan data, penentuan keputusan RL, hingga pembaruan jadwal otomatis ke Control Service. Sebelum memulai pengujian layanan tambahan harus dikonfigurasi terlebih dahulu untuk detail

konfigurasi terdapat pada Lampiran K.1 untuk konfigurasi *Environment Variable* dan berlanjut pada Lampiran K.2 untuk setup service.

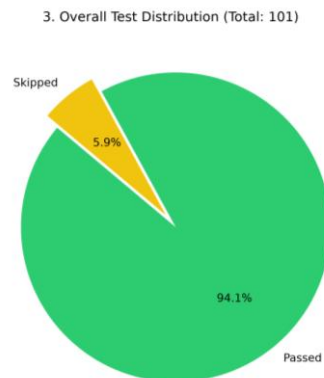
Pada pengujian model-control menampilkan log kontrol adaptif sebagai berikut:

```
model-control-1 | cache metrics: {"T_in": {"value": 36.60000229, "age_s": 15.5},  
"H_in": {"value": 36.79999924, "age_s": 2.7}, "T_out": {"value": 32.20000076,  
"age_s": 3.8}, "H_out": {"value": 45.5, "age_s": 4.5}, "EC": {"value": 1.5, "age_s":  
Infinity}, "pH": {"value": 6.5, "age_s": Infinity}, "T_nut": {"value": 25.0,  
"age_s": Infinity}}  
model-control-1 | state=[46.2, 0.25, 30.0, 36.79999924, 30.0, 45.5, 1.5, 6.5, 25.0,  
0.0]  
model-control-1 | INFO:httpx:HTTP Request: POST http://model-  
controller:8080/predict "HTTP/1.1 200 OK"  
model-control-1 | tick skipped cycle_done=False pending=True elapsed=383.7s  
cycle=780
```

Pengujian ini membuktikan kontrol adaptif telah bekerja metadata visual dari MinIO berhasil digabung dengan parameter sensor, diteruskan ke algoritma RL, dan keputusan durasi *misting/valve* diterapkan secara otomatis ke Control Service. Interval penjadwalan diperbarui secara adaptif (terobservasi di log & UI) tanpa menginterupsi siklus penyemprotan fisik yang sedang berjalan.

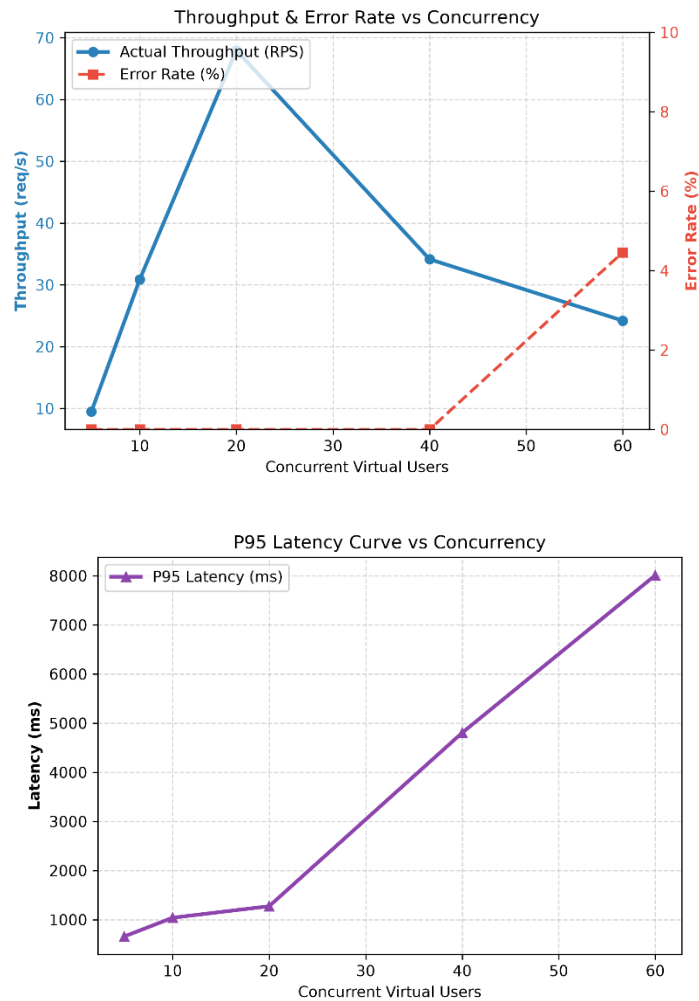
4.4.2 Pengujian Unit Test Fitur Layanan

Pengujian unit test mencakup seluruh 13 microservices beserta modul AI/ML dan Stream Service dengan total **101** fitur yang diuji:



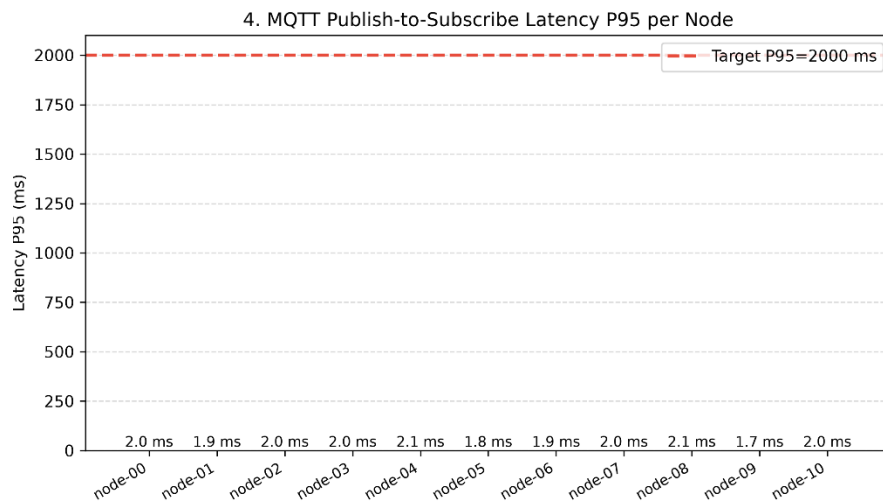
Secara keseluruhan semua fitur berjalan normal dan lulus dan siap digunakan untuk pengujian selanjutnya.

4.4.3 Pengujian Stress Test

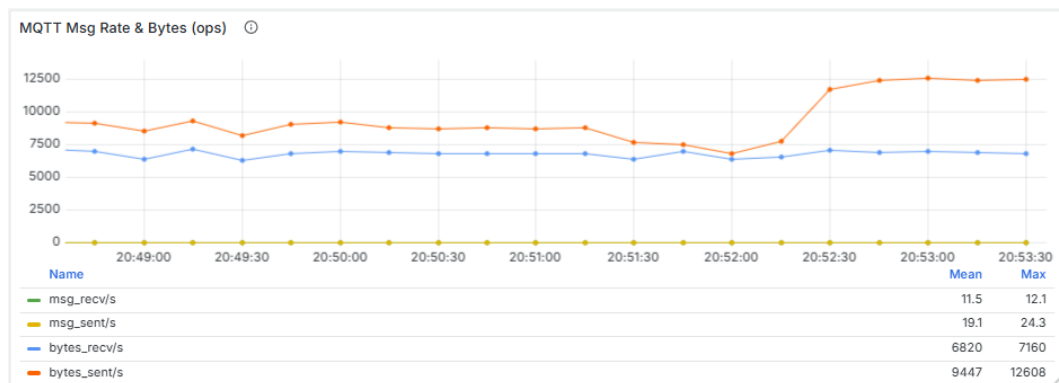


Hasil pengujian menunjukkan grafik *throughput* terhadap konkurensi naik cepat hingga mencapai puncak **68 RPS** pada Level 3 (20 users) dengan Latensi P95 **1275 ms**. Kemudian turun drastis **34.2 RPS** Sedangkan error baru muncul setelah 60 user sebesar di **4,5%**.

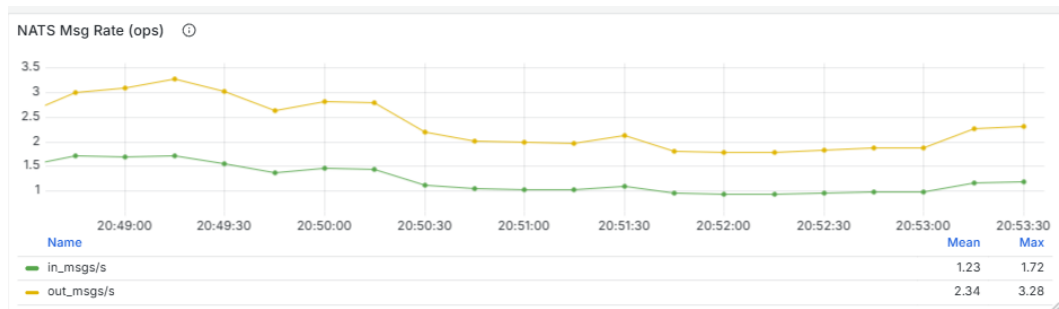
Uji beban di sisi ingesti telemetry diuji dengan beban 10 node dalam 5 module sehingga setiap module memiliki dua node.



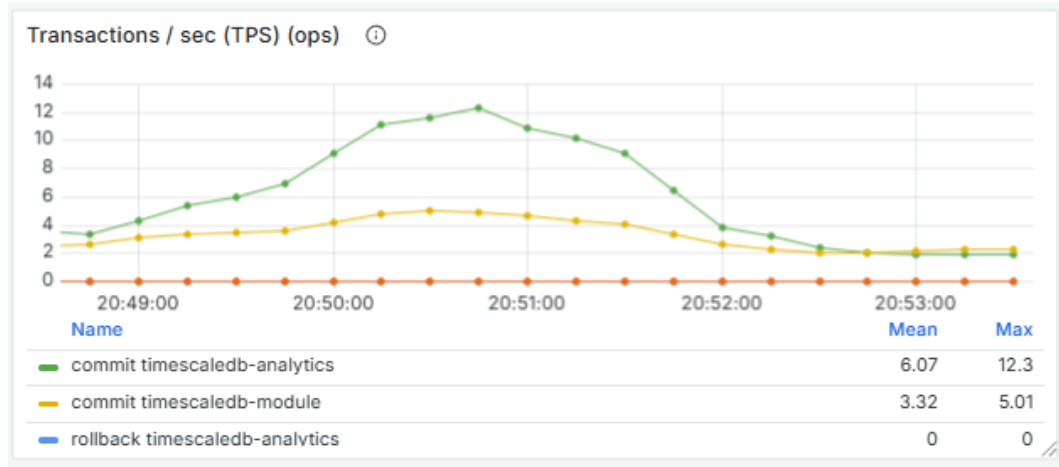
Berdasarkan hasil pengujian latensi transmisi *publish-subscribe* yang dilakukan secara simultan menggunakan **empat node**, diperoleh rata-rata latensi P95 yang sangat rendah, yaitu **2.1 ms** dengan batas toleransi maksimum yaitu **2000 ms** (2 detik).



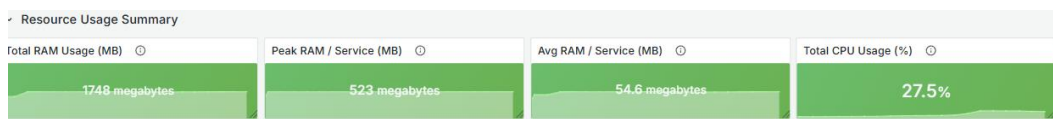
Hasil pengujian throughput MQTT pada broker menunjukkan bahwa rata – rata throughput yang diterima broker adalah **11.5 msg/s**. Ini sesuai dengan konfigurasi setiap node yang mengirimkan telemetri setiap 1 detik sekali.



Berdasarkan grafik pengujian tersebut, rata-rata laju ingesti data stabil berada pada angka **1,2 msg/s** dengan beban puncak maksimum mencapai **1,72 msg/s**.

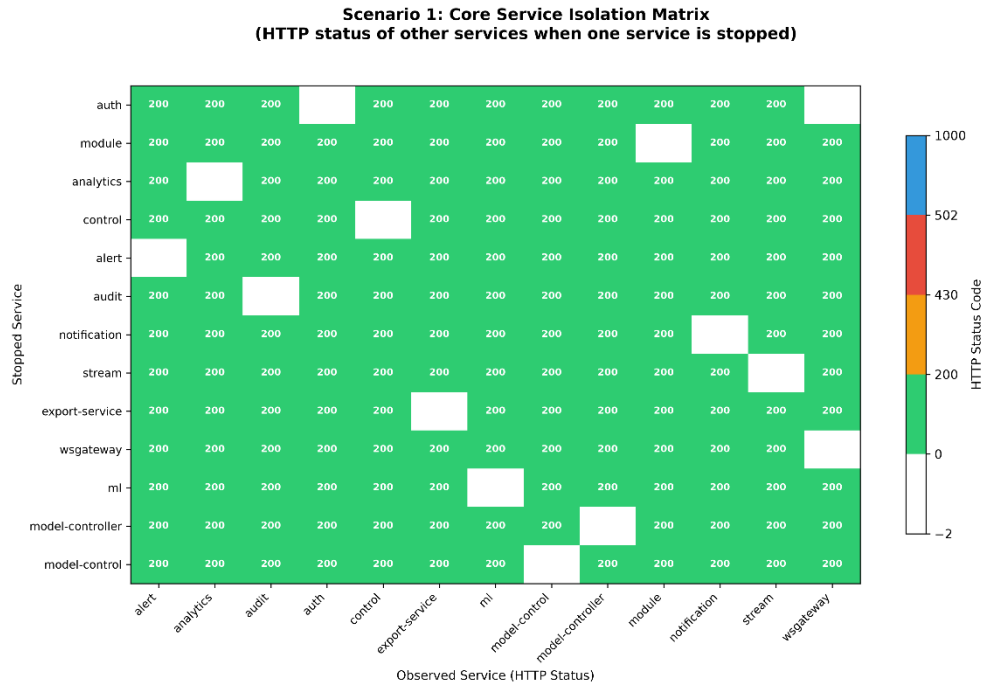


Rata-rata laju transaksi komit pada basis data TimescaleDB oleh *Module Service* dalam jendela waktu 5 menit terakhir, dengan nilai rata-rata sebesar **3,32 tps** dan batas puncak maksimum **5,01 tps**. Untuk Analytics Service nilai rata-rata **6.07 tps** dan batas puncak maksimum **12.3 tps**.



Selama fase pengujian beban, utilisasi sumber daya pada server *backend* menunjukkan efisiensi yang sangat tinggi dengan beban CPU puncak mencapai **27.5%** dan total konsumsi RAM keseluruhan sebesar **1.7 GB**. Di tingkat kontainer, rata-rata konsumsi memori per layanan mikro (*microservice*) berada pada angka yang sangat ringan, yaitu **54.6 MB**, dengan batas konsumsi puncak maksimum per layanan paling intensif terbatas sebesar **523 MB**.

4.4.4 Pengujian Resilience Chaos Test



Secara keseluruhan, pengujian isolasi kegagalan pada 13 *core microservices* menunjukkan ketahanan sistem yang sangat baik. Seluruh skenario *chaos engineering* berhasil dilalui dengan kelulusan 100%, membuktikan bahwa arsitektur sistem mampu mencegah efek kegagalan berantai (*cascade failure*) dan menjaga ketersediaan layanan meskipun terjadi gangguan pada komponen lain.

4.5 Analisis Komparatif dan Pembahasan

Bagian ini membandingkan data empiris dari **Tahap I** dan **Tahap II** untuk menganalisis dampak modularitas arsitektur terhadap performa dan keandalan sistem.

4.5.1 Analisis Kinerja Throughput Latensi dan Resource

Parameter Pengujian	Tahap I	Tahap II
Beban Ideal User	20 User	20 User
Throughput Puncak	51.3 RPS	68 RPS
Latensi P95 API	1482 ms	1275 ms.

Tingkat Kesalahan (Error Rate)	2,1%.	4,5%.
Latensi MQTT	1.75 ms	2.1 ms
Troughput MQTT	4.26 msg/s	11.5 msg/s
Laju Ingesti NATS	1,76 - 2,99 msg/s	1,2 - 1,72 msg/s
TimescaleDB Transaction	3,46 - 4,04 tps (module) 2.44 - 3.03 tps (analytics)	3.32 - 5.01 tps (module) 6.07 - 12.3 tps (analytics)
Total RAM	800 MB	1700 MB
Average RAM/Service	28.1 MB	54.6 MB
Peak RAM/Service	200 MB	523 MB
Peak CPU	9.69%	27.5%

Pengujian menunjukkan bahwa performa REST API Gateway mencapai titik ideal pada beban **20 users** dengan peningkatan throughput puncak dari **51,3 RPS** menjadi **68 RPS** serta penurunan latensi P95 dari **1482 ms** ke **1275 ms**, meskipun diiringi kenaikan error rate tipis dari **2,1%** menjadi **4,5%**. Di sisi ingesti telemetry, sistem terbukti sangat responsif dan andal dengan latensi MQTT yang stabil di kisaran **1,75–2,1 ms**, peningkatan throughput MQTT dari **4,26 msg/s** ke **11,5 msg/s**, serta lonjakan pemrosesan transaksi TimescaleDB Analytics hingga **12,3 tps**. Kenaikan skala pemrosesan ini diimbangi oleh utilisasi sumber daya server yang sangat efisien, di mana konsumsi RAM total meningkat dari **800 MB** ke **1700 MB** dan peak CPU berada pada level aman **27,5%**.

4.5.2 Pengaruh Penambahan Node

Pelipatgandaan perangkat fisik dari 4 node (1 modul) menjadi 10 node (5 modul) meningkatkan volume pesan telemetry secara signifikan tanpa menimbulkan *bottleneck* pada sistem. Berikut adalah ringkasan perubahan kinerja dari Tahap I ke Tahap II:

1. **Peningkatan *Throughput* MQTT:** Naik drastis dari **4,26 msg/s menjadi 11,5 msg/s** (+170%), mencerminkan efisiensi transmisi data *edge* saat beban perangkat bertambah.
2. **Lonjakan Transaksi Analitik TimescaleDB:** Laju transaksi *Analytics Service* melonjak dari **2,44–3,03 tps menjadi 6,07–12,3 tps** (naik 2x hingga 4x lipat), mengindikasikan fitur *hypertable partitioning* bekerja lebih efisien pada volume data tinggi.
3. **Stabilitas Latensi MQTT:** Latensi transmisi naik tipis dari **1,75 ms menjadi 2,1 ms** (+0,35 ms / +20%), namun tetap sangat cepat dan memenuhi ambang batas toleransi.
4. **Penyesuaian *Ingest Rate* NATS:** Laju ingesti NATS berubah dari **1,76–2,99 msg/s menjadi 1,2–1,72 msg/s** akibat perubahan pola *batching* internal broker.
5. **Stabilitas Transaksi Modul TimescaleDB:** Transaksi *Module Service* relatif stabil pada rentang **3,32–5,01 tps**.

Dukungan arsitektur *asynchronous batching* pada NATS JetStream dan *hypertable partitioning* TimescaleDB memastikan transmisi data berjalan lancar. Hal ini dibuktikan oleh latensi pembaruan telemetri ke dasbor WebSocket yang tetap terjaga di rata-rata **~240 ms** ($P_{95} < 1,0$ detik)

4.5.3 Pengaruh Penambahan Layanan AI/ML

Berikut ini adalah beberapa pengaruh penambahan Layanan AI/ML:

1. **Melonjaknya *Throughput*:** Kapasitas puncak naik dari **51,3 RPS menjadi 68 RPS** (+32,5%) meskipun layanan AI/ML tidak menerima beban kueri langsung.
2. **Membaiknya Latensi:** Latensi P_{95} turun dari **1482 ms ke 1275 ms** (lebih cepat 207 ms / -14%).

3. **Kenaikan *Error Rate*:** *Error rate* naik dari **2,1% menjadi 4,5%**, namun masih berada pada batas aman (<10%).
4. **Lonjakan *Resource Usage*:** Konsumsi RAM melonjak dari **800 MB ke 1700 MB (+112,5%)** dan *Peak CPU* naik dari **9,69% ke 27,5% (+184%)**.
5. **Arsitektur Tetap *Loose Coupling*:** Layanan AI/ML terbukti bersifat *non-blocking*; beban tambahan tidak menginterupsi siklus operasional utama maupun menyebabkan kegagalan beruntun (*cascade failure*).

4.5.4 Keandalan Sistem Menangani Service yang Gagal

Berdasarkan hasil 6 skenario chaos engineering pada Tahap I dan II, kesimpulan keseluruhan sistem dalam menangani service yang gagal adalah:

1. Isolasi Kegagalan Berhasil Dijamin Tidak terjadi cascade failure dalam skenario apapun. Ketika satu layanan dihentikan — baik core service, database, broker, gateway, maupun layanan AI/ML — layanan lain tetap berjalan dan merespon dengan benar. Tiga kegagalan minor yang muncul di Tahap I hanya terbatas pada Module Service saat dependency sekunder (Redis dan pemulihan DB) terganggu, namun tidak merembet ke service lain.
2. Recovery Mandiri (Self-Healing) Semua layanan yang dihentikan pulih secara otomatis tanpa intervensi manual:
 - a. NATS: ~7 detik
 - b. Module Service: ~6 detik
 - c. Kong: ~188 detik
3. Arsitektur yang Memungkinkan Isolasi Tiga mekanisme utama bekerja bersama:
 - a. Database-per-Service: Kegagalan database satu layanan tidak memengaruhi layanan lain

- b. NATS JetStream (async): Pesan tetap tersimpan saat subscriber gagal, dilanjutkan setelah recovery
 - c. Kong Gateway: Memfilter request ke service yang sehat, mencegah thundering herd
4. Skalabilitas Teruji Sistem tetap menampung penambahan layanan (dari 10 menjadi 13 services) dan perangkat fisik (dari 4 menjadi 10 node) tanpa mengorbankan stabilitas. Pengujian Tahap II mencapai 100% lulus pada skala penuh.

4.6 Kesimpulan Awal

Berdasarkan seluruh rangkaian implementasi, pengujian bertahap, dan analisis komparatif yang telah dilakukan, dapat ditarik kesimpulan awal sebagai berikut:

1. Mekanisme konfigurasi via antarmuka Captive Portal LittleFS terbukti memfasilitasi integrasi berberapa sensor dan aktuator termasuk sensor I2C INA219 baru secara instan tanpa kompilasi ulang kode.
2. Alur autentikasi JWT, pembuatan modul, *auto-discovery pairing*, *tag mapping* visual, ingesti data TimescaleDB, visualisasi WebSocket dan kontrol aktuator berfungsi sebagaimana mestinya.
3. Meskipun ekspansi sistem memicu penurunan throughput pada beban penuh, arsitektur modular terbukti tetap efisien pada kondisi operasi normal.
4. Arsitektur *loose coupling* berbasis NATS JetStream dan Kong Gateway menjamin isolasi kegagalan antar bounded context tidak ada *cascade failure* ke service lain.

BAB V

KESIMPULAN

5.1 Kesimpulan

Dari serangkaian proses perancangan, implementasi, hingga validasi sistem pemantauan dan kontrol lingkungan aeroponik berbasis arsitektur microservice, ditarik kesimpulan sebagai berikut:

1. **Sistem Microservice Modular:** Sistem berhasil dibangun dengan 13 layanan backend mandiri, masing-masing memiliki database terisolasi, kontrak komunikasi (REST via Kong, NATS JetStream, MQTT), dan deployment tersendiri. Prinsip single responsibility dan bounded context dijunjung, memungkinkan pengembangan dan deploy independen tanpa mengganggu layanan lain.
2. **Firmware ESP32 Modular:** Firmware berhasil diimplementasikan dengan pola factory pattern dan protocol registry yang memungkinkan penambahan sensor dan protokol baru secara dinamis melalui config.json tanpa kompilasi ulang. Pembuktian empiris melalui penambahan sensor I2C (INA) yang dilakukan melalui antarmuka.
3. **Keunggulan Modularitas Terbukti Secara Empiris:** Melalui 6 skenario chaos engineering pada Tahap I dan II, sistem membuktikan isolasi kegagalan absolut (zero cascade failure) dengan 100% lulus.
4. **Integrasi Model Kontrol TD3:** Model TD3 berhasil dikembangkan from scratch dengan mean episode reward 6,671 (target >5.000). Integrasi dilakukan secara loose coupling melalui dua layanan khusus (model-control dan model-controller). Integrasi ini berhasil memperbarui interval kontrol untuk setiap siklus.
5. **Evaluasi Kuantitatif:** Stress test menunjukkan kapasitas tinggi pada beban ideal—450,9 RPS dengan P95 159,3 ms dan error rate 0,0% serta isolasi kegagalan penuh. Pada skala penuh 10 node, P95 maksimum 1183,2 ms

pada Level 3 (51,3 RPS) dengan knee point pada Level 4, mengindikasikan kebutuhan optimasi bottleneck untuk beban ekstrem.

5.2 Saran

Berdasarkan temuan penelitian dan keterbatasan yang teridentifikasi, berikut adalah saran untuk pengembangan sistem lebih lanjut:

1. **Pelengkapan Protokol Sensor dan Aktuator pada Firmware:** Firmware ESP32 saat ini sudah mendukung GPIO Digital/Analog, Modbus RTU, dan I2C. Disarankan untuk melengkapi dukungan protokol lain yang umum digunakan di pertanian presisi, seperti 1-Wire (DS18B20), CAN bus, dan PWM/ADC khusus untuk aktuator, agar sistem dapat mengintegrasikan beragam jenis sensor dan aktuator tanpa perlu modifikasi arsitektur inti.
2. **Konfigurasi Terpadu dalam Satu Dashboard:** Saat ini konfigurasi firmware dilakukan melalui Captive Web Portal terpisah. Disarankan untuk mengintegrasikan konfigurasi firmware seperti penambahan sensor dan pin mapping langsung ke dalam dashboard utama, sehingga pengguna dapat mengelola seluruh sistem (node, sensor, aktuator, dan jadwal) dari satu antarmuka tanpa perlu beralih ke portal perangkat.

DAFTAR PUSTAKA

LAMPIRAN